

For an input shape `(5,3)`, each row represents a lagged version of the three variables. The `nn.RNN()` layer also expects the first row of each observation to be earliest in time, so we must reverse the current order. Hence we loop over `range(5,0,-1)` below, which is an example of using a `slice()` to index iterable objects. The general notation is `start:end:step`.

```
In [99]: ordered_cols = []
        for lag in range(5,0,-1):
            for col in cols:
                ordered_cols.append('{0}_{1}'.format(col, lag))
        X = X.reindex(columns=ordered_cols)
        X.columns
```

```
Out[99]: Index(['DJ_return_5', 'log_volume_5', 'log_volatility_5',
               'DJ_return_4', 'log_volume_4', 'log_volatility_4',
               'DJ_return_3', 'log_volume_3', 'log_volatility_3',
               'DJ_return_2', 'log_volume_2', 'log_volatility_2',
               'DJ_return_1', 'log_volume_1', 'log_volatility_1'],
              dtype='object')
```

We now reshape the data.

```
In [100]: X_rnn = X.to_numpy().reshape((-1,5,3))
          X_rnn.shape
```

```
Out[100]: (6046, 5, 3)
```

By specifying the first size as -1, `numpy.reshape()` deduces its size based on the remaining arguments.

Now we are ready to proceed with the RNN, which uses 12 hidden units, and 10% dropout. After passing through the RNN, we extract the final time point as `val[:,-1]` in `forward()` below. This gets passed through a 10% dropout and then flattened through a linear layer.

```
In [101]: class NYSEModel(nn.Module):
        def __init__(self):
            super(NYSEModel, self).__init__()
            self.rnn = nn.RNN(3,
                              12,
                              batch_first=True)
            self.dense = nn.Linear(12, 1)
            self.dropout = nn.Dropout(0.1)
        def forward(self, x):
            val, h_n = self.rnn(x)
            val = self.dense(self.dropout(val[:,-1]))
            return torch.flatten(val)
        nyse_model = NYSEModel()
```

We fit the model in a similar fashion to previous networks. We supply the `fit` function with test data as validation data, so that when we monitor its progress and plot the history function we can see the progress on the test data. Of course we should not use this as a basis for early stopping, since then the test performance would be biased.

We form the training dataset similar to our `Hitters` example.

```
In [102]: datasets = []
for mask in [train, ~train]:
    X_rnn_t = torch.tensor(X_rnn[mask].astype(np.float32))
    Y_t = torch.tensor(Y[mask].astype(np.float32))
    datasets.append(TensorDataset(X_rnn_t, Y_t))
nyse_train, nyse_test = datasets
```

Following our usual pattern, we inspect the summary.

```
In [103]: summary(nyse_model,
                 input_data=X_rnn_t,
                 col_names=['input_size',
                           'output_size',
                           'num_params'])
```

```
Out [103]: =====
Layer (type:depth-idx)   Input Shape      Output Shape     Param #
=====
NYSEModel                [1770, 5, 3]    [1770]           --
  RNN: 1-1                [1770, 5, 3]    [1770, 5, 12]    204
  Dropout: 1-2            [1770, 12]       [1770, 12]       --
  Linear: 1-3              [1770, 12]       [1770, 1]        13
=====
Total params: 217
Trainable params: 217
```

We again put the two datasets into a data module, with a batch size of 64.

```
In [104]: nyse_dm = SimpleDataModule(nyse_train,
                                     nyse_test,
                                     num_workers=min(4, max_num_workers),
                                     validation=nyse_test,
                                     batch_size=64)
```

We run some data through our model to be sure the sizes match up correctly.

```
In [105]: for idx, (x, y) in enumerate(nyse_dm.train_dataloader()):
           out = nyse_model(x)
           print(y.size(), out.size())
           if idx >= 2:
               break
```

```
torch.Size([64]) torch.Size([64])
torch.Size([64]) torch.Size([64])
torch.Size([64]) torch.Size([64])
```

We follow our previous example for setting up a trainer for a regression problem, requesting the R^2 metric to be computed at each epoch.

```
In [106]: nyse_optimizer = RMSprop(nyse_model.parameters(),
                                   lr=0.001)
nyse_module = SimpleModule.regression(nyse_model,
                                     optimizer=nyse_optimizer,
                                     metrics={'r2': R2Score()})
```

Fitting the model should by now be familiar. The results on the test data are very similar to the linear AR model.

```
In [107]: nyse_trainer = Trainer(deterministic=True,
                                max_epochs=200,
                                callbacks=[ErrorTracker()])
nyse_trainer.fit(nyse_module,
                 datamodule=nyse_dm)
nyse_trainer.test(nyse_module,
                  datamodule=nyse_dm)
```

```
Out[107]: [{'test_loss': 0.6141, 'test_r2': 0.4172}]
```

We could also fit a model without the `nn.RNN()` layer by just using a `nn.Flatten()` layer instead. This would be a nonlinear AR model. If in addition we excluded the hidden layer, this would be equivalent to our earlier linear AR model.

Instead we will fit a nonlinear AR model using the feature set `X_day` that includes the `day_of_week` indicators. To do so, we must first create our test and training datasets and a corresponding data module. This may seem a little burdensome, but is part of the general pipeline for `torch`.

```
In [108]: datasets = []
for mask in [train, ~train]:
    X_day_t = torch.tensor(
        np.asarray(X_day[mask]).astype(np.float32))
    Y_t = torch.tensor(np.asarray(Y[mask]).astype(np.float32))
    datasets.append(TensorDataset(X_day_t, Y_t))
day_train, day_test = datasets
```

Creating a data module follows a familiar pattern.

```
In [109]: day_dm = SimpleDataModule(day_train,
                                     day_test,
                                     num_workers=min(4, max_num_workers),
                                     validation=day_test,
                                     batch_size=64)
```

We build a `NonLinearARModel()` that takes as input the 20 features and a hidden layer with 32 units. The remaining steps are familiar.

```
In [110]: class NonLinearARModel(nn.Module):
            def __init__(self):
                super(NonLinearARModel, self).__init__()
                self._forward = nn.Sequential(nn.Flatten(),
                                                nn.Linear(20, 32),
                                                nn.ReLU(),
                                                nn.Dropout(0.5),
                                                nn.Linear(32, 1))

            def forward(self, x):
                return torch.flatten(self._forward(x))
```

[illegible]

We continue with the usual training steps, fit the model, and evaluate the test error. We see the test R^2 is a slight improvement over the linear AR model that also includes `day_of_week`.

```
In [112]: nl_trainer = Trainer(deterministic=True,
                               max_epochs=20,
                               callbacks=[ErrorTracker()])
nl_trainer.fit(nl_module, datamodule=day_dm)
nl_trainer.test(nl_module, datamodule=day_dm)
```

```
Out[112]: [{'test_loss': 0.5625, 'test_r2': 0.4662}]
```

10.10 Exercises

Conceptual

- Consider a neural network with two hidden layers: $p = 4$ input units, 2 units in the first hidden layer, 3 units in the second hidden layer, and a single output.
 - Draw a picture of the network, similar to Figures 10.1 or 10.4.
 - Write out an expression for $f(X)$, assuming ReLU activation functions. Be as explicit as you can!
 - Now plug in some values for the coefficients and write out the value of $f(X)$.
 - How many parameters are there?
- Consider the *softmax* function in (10.13) (see also (4.13) on page 145) for modeling multinomial probabilities.
 - In (10.13), show that if we add a constant c to each of the z_ℓ , then the probability is unchanged.
 - In (4.13), show that if we add constants c_j , $j = 0, 1, \dots, p$, to each of the corresponding coefficients for each of the classes, then the predictions at any new point x are unchanged.

This shows that the softmax function is *over-parametrized*. However, regularization and SGD typically constrain the solutions so that this over-parametrized is not a problem.

- Show that the negative multinomial log-likelihood (10.14) is equivalent to the negative log of the likelihood expression (4.5) when there are $M = 2$ classes.
- Consider a CNN that takes in 32×32 grayscale images and has a single convolution layer with three 5×5 convolution filters (without boundary padding).
 - Draw a sketch of the input and first hidden layer similar to Figure 10.8.

- (b) How many parameters are in this model?
 - (c) Explain how this model can be thought of as an ordinary feed-forward neural network with the individual pixels as inputs, and with constraints on the weights in the hidden units. What are the constraints?
 - (d) If there were no constraints, then how many weights would there be in the ordinary feed-forward neural network in (c)?
5. In Table 10.2 on page 426, we see that the ordering of the three methods with respect to mean absolute error is different from the ordering with respect to test set R^2 . How can this be?

Applied

6. Consider the simple function $R(\beta) = \sin(\beta) + \beta/10$.
- (a) Draw a graph of this function over the range $\beta \in [-6, 6]$.
 - (b) What is the derivative of this function?
 - (c) Given $\beta^0 = 2.3$, run gradient descent to find a local minimum of $R(\beta)$ using a learning rate of $\rho = 0.1$. Show each of $\beta^0, \beta^1, \dots$ in your plot, as well as the final answer.
 - (d) Repeat with $\beta^0 = 1.4$.
7. Fit a neural network to the **Default** data. Use a single hidden layer with 10 units, and dropout regularization. Have a look at Labs 10.9.1–10.9.2 for guidance. Compare the classification performance of your model with that of linear logistic regression.
8. From your collection of personal photographs, pick 10 images of animals (such as dogs, cats, birds, farm animals, etc.). If the subject does not occupy a reasonable part of the image, then crop the image. Now use a pretrained image classification CNN as in Lab 10.9.4 to predict the class of each of your images, and report the probabilities for the top five predicted classes for each image.
9. Fit a lag-5 autoregressive model to the **NYSE** data, as described in the text and Lab 10.9.6. Refit the model with a 12-level factor representing the month. Does this factor improve the performance of the model?
10. In Section 10.9.6, we showed how to fit a linear AR model to the **NYSE** data using the `LinearRegression()` function. However, we also mentioned that we can “flatten” the short sequences produced for the RNN model in order to fit a linear AR model. Use this latter approach to fit a linear AR model to the **NYSE** data. Compare the test R^2 of this linear AR model to that of the linear AR model that we fit in the lab. What are the advantages/disadvantages of each approach?
11. Repeat the previous exercise, but now fit a nonlinear AR model by “flattening” the short sequences produced for the RNN model.

12. Consider the RNN fit to the **NYSE** data in Section 10.9.6. Modify the code to allow inclusion of the variable **day_of_week**, and fit the RNN. Compute the test R^2 .
13. Repeat the analysis of Lab 10.9.5 on the **IMDb** data using a similarly structured neural network. We used 16 hidden units at each of two hidden layers. Explore the effect of increasing this to 32 and 64 units per layer, with and without 30% dropout regularization.



Survival Analysis and Censored Data

In this chapter, we will consider the topics of *survival analysis* and *censored data*. These arise in the analysis of a unique kind of outcome variable: the *time until an event occurs*.

survival
analysis
censored
data

For example, suppose that we have conducted a five-year medical study, in which patients have been treated for cancer. We would like to fit a model to predict patient survival time, using features such as baseline health measurements or type of treatment. At first pass, this may sound like a regression problem of the kind discussed in Chapter 3. But there is an important complication: hopefully some or many of the patients have survived until the end of the study. Such a patient's survival time is said to be *censored*: we know that it is at least five years, but we do not know its true value. We do not want to discard this subset of surviving patients, as the fact that they survived at least five years amounts to valuable information. However, it is not clear how to make use of this information using the techniques covered thus far in this textbook.

Though the phrase “survival analysis” evokes a medical study, the applications of survival analysis extend far beyond medicine. For example, consider a company that wishes to model *churn*, the process by which customers cancel subscription to a service. The company might collect data on customers over some time period, in order to model each customer's time to cancellation as a function of demographics or other predictors. However, presumably not all customers will have canceled their subscription by the end of this time period; for such customers, the time to cancellation is censored.

In fact, survival analysis is relevant even in application areas that are unrelated to time. For instance, suppose we wish to model a person's weight as a function of some covariates, using a dataset with measurements for a large number of people. Unfortunately, the scale used to weigh those people is unable to report weights above a certain number. Then, any weights that

exceed that number are censored. The survival analysis methods presented in this chapter could be used to analyze this dataset.

Survival analysis is a very well-studied topic within statistics, due to its critical importance in a variety of applications, both in and out of medicine. However, it has received relatively little attention in the machine learning community.

11.1 Survival and Censoring Times

For each individual, we suppose that there is a true *survival time*, T , as well as a true *censoring time*, C . (The survival time is also known as the *failure time* or the *event time*.) The survival time represents the time at which the event of interest occurs: for instance, the time at which the patient dies, or the customer cancels his or her subscription. By contrast, the censoring time is the time at which censoring occurs: for example, the time at which the patient drops out of the study or the study ends.

survival time
censoring
time
failure time
event time

We observe either the survival time T or else the censoring time C . Specifically, we observe the random variable

$$Y = \min(T, C). \quad (11.1)$$

In other words, if the event occurs before censoring (i.e. $T < C$) then we observe the true survival time T ; however, if censoring occurs before the event ($T > C$) then we observe the censoring time. We also observe a status indicator,

$$\delta = \begin{cases} 1 & \text{if } T \leq C \\ 0 & \text{if } T > C. \end{cases}$$

Thus, $\delta = 1$ if we observe the true survival time, and $\delta = 0$ if we instead observe the censoring time.

Now, suppose we observe n (Y, δ) pairs, which we denote as $(y_1, \delta_1), \dots, (y_n, \delta_n)$. Figure 11.1 displays an example from a (fictitious) medical study in which we observe $n = 4$ patients for a 365-day follow-up period. For patients 1 and 3, we observe the time to event (such as death or disease relapse) $T = t_i$. Patient 2 was alive when the study ended, and patient 4 dropped out of the study, or was “lost to follow-up”; for these patients we observe $C = c_i$. Therefore, $y_1 = t_1$, $y_3 = t_3$, $y_2 = c_2$, $y_4 = c_4$, $\delta_1 = \delta_3 = 1$, and $\delta_2 = \delta_4 = 0$.

11.2 A Closer Look at Censoring

In order to analyze survival data, we need to make some assumptions about *why* censoring has occurred. For instance, suppose that a number of patients drop out of a cancer study early because they are very sick. An analysis that does not take into consideration the reason why the patients dropped out will likely overestimate the true average survival time. Similarly, suppose that males who are very sick are more likely to drop out of the study than

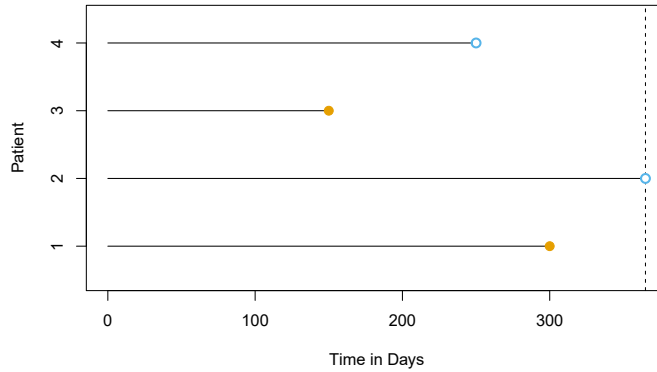


FIGURE 11.1. Illustration of censored survival data. For patients 1 and 3, the event was observed. Patient 2 was alive when the study ended. Patient 4 dropped out of the study.

females who are very sick. Then a comparison of male and female survival times may wrongly suggest that males survive longer than females.

In general, we need to assume that the censoring mechanism is *independent*: conditional on the features, the event time T is independent of the censoring time C . The two examples above violate the assumption of independent censoring. Typically, it is not possible to determine from the data itself whether the censoring mechanism is independent. Instead, one has to carefully consider the data collection process in order to determine whether independent censoring is a reasonable assumption. In the remainder of this chapter, we will assume that the censoring mechanism is independent.¹

In this chapter, we focus on *right censoring*, which occurs when $T \geq Y$, i.e. the true event time T is at least as large as the observed time Y . (Notice that $T \geq Y$ is a consequence of (11.1). Right censoring derives its name from the fact that time is typically displayed from left to right, as in Figure 11.1.) However, other types of censoring are possible. For instance, in *left censoring*, the true event time T is less than or equal to the observed time Y . For example, in a study of pregnancy duration, suppose that we survey patients 250 days after conception, when some have already had their babies. Then we know that for those patients, pregnancy duration is less than 250 days. More generally, *interval censoring* refers to the setting in which we do not know the exact event time, but we know that it falls in some interval. For instance, this setting arises if we survey patients once per week in order to determine whether the event has occurred. While left censoring and interval censoring can be accommodated using variants of the ideas presented in this chapter, in what follows we focus specifically on right censoring.

¹The assumption of independent censoring can be relaxed somewhat using the notion of *non-informative censoring*; however, the definition of non-informative censoring is too technical for this book.

11.3 The Kaplan–Meier Survival Curve

The *survival curve*, or *survival function*, is defined as

$$S(t) = \Pr(T > t). \quad (11.2)$$

survival
curve
survival
function

This decreasing function quantifies the probability of surviving past time t . For example, suppose that a company is interested in modeling customer churn. Let T represent the time that a customer cancels a subscription to the company’s service. Then $S(t)$ represents the probability that a customer cancels later than time t . The larger the value of $S(t)$, the less likely that the customer will cancel before time t .

In this section, we will consider the task of estimating the survival curve. Our investigation is motivated by the **BrainCancer** dataset, which contains the survival times for patients with primary brain tumors undergoing treatment with stereotactic radiation methods.² The predictors are **gtv** (gross tumor volume, in cubic centimeters); **sex** (male or female); **diagnosis** (meningioma, LG glioma, HG glioma, or other); **loc** (the tumor location: either infratentorial or supratentorial); **ki** (Karnofsky index); and **stereo** (stereotactic method: either stereotactic radiosurgery or fractionated stereotactic radiotherapy, abbreviated as SRS and SRT, respectively). Only 53 of the 88 patients were still alive at the end of the study.

Now, we consider the task of estimating the survival curve (11.2) for these data. To estimate $S(20) = \Pr(T > 20)$, the probability that a patient survives for at least $t = 20$ months, it is tempting to simply compute the proportion of patients who are known to have survived past 20 months, i.e. the proportion of patients for whom $Y > 20$. This turns out to be 48/88, or approximately 55%. However, this does not seem quite right, since Y and T represent different quantities. In particular, 17 of the 40 patients who did not survive to 20 months were actually censored, and this analysis implicitly assumes that $T < 20$ for all of those censored patients; of course, we do not know whether that is true.

Alternatively, to estimate $S(20)$, we could consider computing the proportion of patients for whom $Y > 20$, out of the 71 patients who were *not* censored by time $t = 20$; this comes out to 48/71, or approximately 68%. However, this is not quite right either, since it amounts to completely ignoring the patients who were censored before time $t = 20$, even though the *time* at which they are censored is potentially informative. For instance, a patient who was censored at time $t = 19.9$ likely would have survived past $t = 20$ had he or she not been censored.

We have seen that estimating $S(t)$ is complicated by the presence of censoring. We now present an approach to overcome these challenges. We let $d_1 < d_2 < \dots < d_K$ denote the K unique death times among the non-censored patients, and we let q_k denote the number of patients who died at time d_k . For $k = 1, \dots, K$, we let r_k denote the number of patients alive

²This dataset is described in the following paper: Selingerová et al. (2016) Survival of patients with primary brain tumors: Comparison of two statistical approaches. PLoS One, 11(2):e0148733.

and in the study just before d_k ; these are the *at risk* patients. The set of patients that are at risk at a given time are referred to as the *risk set*.

By the law of total probability,³

$$\begin{aligned}\Pr(T > d_k) &= \Pr(T > d_k | T > d_{k-1}) \Pr(T > d_{k-1}) \\ &\quad + \Pr(T > d_k | T \leq d_{k-1}) \Pr(T \leq d_{k-1}).\end{aligned}$$

The fact that $d_{k-1} < d_k$ implies that $\Pr(T > d_k | T \leq d_{k-1}) = 0$ (it is impossible for a patient to survive past time d_k if he or she did not survive until an earlier time d_{k-1}). Therefore,

$$S(d_k) = \Pr(T > d_k) = \Pr(T > d_k | T > d_{k-1}) \Pr(T > d_{k-1}).$$

Plugging in (11.2) again, we see that

$$S(d_k) = \Pr(T > d_k | T > d_{k-1}) S(d_{k-1}).$$

This implies that

$$S(d_k) = \Pr(T > d_k | T > d_{k-1}) \times \cdots \times \Pr(T > d_2 | T > d_1) \Pr(T > d_1).$$

We now must simply plug in estimates of each of the terms on the right-hand side of the previous equation. It is natural to use the estimator

$$\widehat{\Pr}(T > d_j | T > d_{j-1}) = (r_j - q_j) / r_j,$$

which is the fraction of the risk set at time d_j who survived past time d_j . This leads to the *Kaplan–Meier estimator* of the survival curve:

$$\widehat{S}(d_k) = \prod_{j=1}^k \left(\frac{r_j - q_j}{r_j} \right). \quad (11.3)$$

For times t between d_k and d_{k+1} , we set $\widehat{S}(t) = \widehat{S}(d_k)$. Consequently, the Kaplan–Meier survival curve has a step-like shape.

The Kaplan–Meier survival curve for the **BrainCancer** data is displayed in Figure 11.2. Each point in the solid step-like curve shows the estimated probability of surviving past the time indicated on the horizontal axis. The estimated probability of survival past 20 months is 71%, which is quite a bit higher than the naive estimates of 55% and 68% presented earlier.

The sequential construction of the Kaplan–Meier estimator — starting at time zero and mapping out the observed events as they unfold in time — is fundamental to many of the key techniques in survival analysis. These include the log-rank test of Section 11.4, and Cox’s proportional hazard model of Section 11.5.2.

³The law of total probability states that for any two events A and B , $\Pr(A) = \Pr(A|B) \Pr(B) + \Pr(A|B^c) \Pr(B^c)$, where B^c is the complement of the event B , i.e. it is the event that B does not hold.

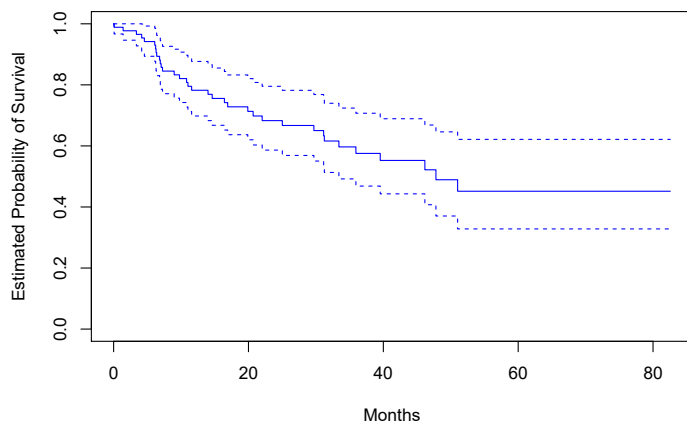


FIGURE 11.2. For the **BrainCancer** data, we display the Kaplan–Meier survival curve (solid curve), along with standard error bands (dashed curves).

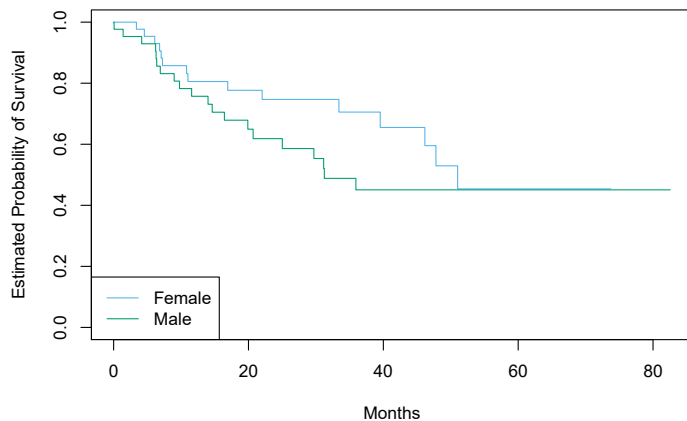


FIGURE 11.3. For the **BrainCancer** data, Kaplan–Meier survival curves for males and females are displayed.

11.4 The Log-Rank Test

We now continue our analysis of the **BrainCancer** data introduced in Section 11.3. We wish to compare the survival of males to that of females. Figure 11.3 shows the Kaplan–Meier survival curves for the two groups. Females seem to fare a little better up to about 50 months, but then the two curves both level off to about 50%. How can we carry out a formal test of equality of the two survival curves?

At first glance, a two-sample t -test seems like an obvious choice: we could test whether the mean survival time among the females equals the mean survival time among the males. But the presence of censoring again creates a complication. To overcome this challenge, we will conduct a *log-rank test*,⁴

log-rank test

	Group 1	Group 2	Total
Died	q_{1k}	q_{2k}	q_k
Survived	$r_{1k} - q_{1k}$	$r_{2k} - q_{2k}$	$r_k - q_k$
Total	r_{1k}	r_{2k}	r_k

TABLE 11.1. Among the set of patients at risk at time d_k , the number of patients who died and survived in each of two groups is reported.

which examines how the events in each group unfold sequentially in time.

Recall from Section 11.3 that $d_1 < d_2 < \dots < d_K$ are the unique death times among the non-censored patients, r_k is the number of patients at risk at time d_k , and q_k is the number of patients who died at time d_k . We further define r_{1k} and r_{2k} to be the number of patients in groups 1 and 2, respectively, who are at risk at time d_k . Similarly, we define q_{1k} and q_{2k} to be the number of patients in groups 1 and 2, respectively, who died at time d_k . Note that $r_{1k} + r_{2k} = r_k$ and $q_{1k} + q_{2k} = q_k$.

At each death time d_k , we construct a 2×2 table of counts of the form shown in Table 11.1. Note that if the death times are unique (i.e. no two individuals die at the same time), then one of q_{1k} and q_{2k} equals one, and the other equals zero.

The main idea behind the log-rank test statistic is as follows. In order to test $H_0 : E(X) = \mu$ for some random variable X , one approach is to construct a test statistic of the form

$$W = \frac{X - \mu}{\sqrt{\text{Var}(X)}}. \quad (11.4)$$

To construct the log-rank test statistic, we compute a quantity that takes exactly the form (11.4), with $X = \sum_{k=1}^K q_{1k}$, where q_{1k} is given in the top left of Table 11.1.

In greater detail, if there is no difference in survival between the two groups, and conditioning on the row and column totals in Table 11.1, the expected value of q_{1k} is

$$\mu_k = \frac{r_{1k}}{r_k} q_k. \quad (11.5)$$

So the expected value of $X = \sum_{k=1}^K q_{1k}$ is $\mu = \sum_{k=1}^K \frac{r_{1k}}{r_k} q_k$. Furthermore, it can be shown⁵ that the variance of q_{1k} is

$$\text{Var}(q_{1k}) = \frac{q_k(r_{1k}/r_k)(1 - r_{1k}/r_k)(r_k - q_k)}{r_k - 1}. \quad (11.6)$$

Though $q_{11}, \dots, q_{1K}$ may be correlated, we nonetheless estimate

$$\text{Var}\left(\sum_{k=1}^K q_{1k}\right) \approx \sum_{k=1}^K \text{Var}(q_{1k}) = \sum_{k=1}^K \frac{q_k(r_{1k}/r_k)(1 - r_{1k}/r_k)(r_k - q_k)}{r_k - 1}. \quad (11.7)$$

⁴The log-rank test is also known as the *Mantel-Haenszel test* or *Cochran-Mantel-Haenszel test*.

⁵For details, see Exercise 7 at the end of this chapter.

Therefore, to compute the log-rank test statistic, we simply proceed as in (11.4), with $X = \sum_{k=1}^K q_{1k}$, making use of (11.5) and (11.7). That is, we calculate

$$W = \frac{\sum_{k=1}^K (q_{1k} - \mu_k)}{\sqrt{\sum_{k=1}^K \text{Var}(q_{1k})}} = \frac{\sum_{k=1}^K \left(q_{1k} - \frac{q_k}{r_k} r_{1k} \right)}{\sqrt{\sum_{k=1}^K \frac{q_k (r_{1k}/r_k)(1-r_{1k}/r_k)(r_k - q_k)}{r_k - 1}}}. \quad (11.8)$$

When the sample size is large, the log-rank test statistic W has approximately a standard normal distribution; this can be used to compute a p -value for the null hypothesis that there is no difference between the survival curves in the two groups.⁶

Comparing the survival times of females and males on the **BrainCancer** data gives a log-rank test statistic of $W = 1.2$, which corresponds to a two-sided p -value of 0.2 using the theoretical null distribution, and a p -value of 0.25 using the permutation null distribution with 1,000 permutations. Thus, we cannot reject the null hypothesis of no difference in survival curves between females and males.

The log-rank test is closely related to Cox's proportional hazards model, which we discuss in Section 11.5.2.

11.5 Regression Models With a Survival Response

We now consider the task of fitting a regression model to survival data. As in Section 11.1, the observations are of the form (Y, δ) , where $Y = \min(T, C)$ is the (possibly censored) survival time, and δ is an indicator variable that equals 1 if $T \leq C$. Furthermore, $X \in \mathbb{R}^p$ is a vector of p features. We wish to predict the true survival time T .

Since the observed quantity Y is positive and may have a long right tail, we might be tempted to fit a linear regression of $\log(Y)$ on X . But as the reader will surely guess, censoring again creates a problem since we are actually interested in predicting T and not Y . To overcome this difficulty, we instead make use of a sequential construction, similar to the constructions of the Kaplan–Meier survival curve in Section 11.3 and the log-rank test in Section 11.4.

11.5.1 The Hazard Function

The *hazard function* or *hazard rate* — also known as the *force of mortality* — is formally defined as hazard
function

$$h(t) = \lim_{\Delta t \rightarrow 0} \frac{\Pr(t < T \leq t + \Delta t | T > t)}{\Delta t}, \quad (11.9)$$

⁶Alternatively, we can estimate the p -value via permutations, using ideas that will be presented in Section 13.5. The permutation distribution is obtained by randomly swapping the labels for the observations in the two groups.

where T is the (unobserved) survival time. It is the death rate in the instant after time t , given survival past that time.⁷ In (11.9), we take the limit as Δt approaches zero, so we can think of Δt as being an extremely tiny number. Thus, more informally, (11.9) implies that

$$h(t) \approx \frac{\Pr(t < T \leq t + \Delta t | T > t)}{\Delta t}$$

for some arbitrarily small Δt .

Why should we care about the hazard function? First of all, it is closely related to the survival curve (11.2), as we will see next. Second, it turns out that a key approach for modeling survival data as a function of covariates relies heavily on the hazard function; we will introduce this approach — Cox's proportional hazards model — in Section 11.5.2.

We now consider the hazard function $h(t)$ in a bit more detail. Recall that for two events A and B , the probability of A given B can be expressed as $\Pr(A | B) = \Pr(A \cap B) / \Pr(B)$, i.e. the probability that A and B both occur divided by the probability that B occurs. Furthermore, recall from (11.2) that $S(t) = \Pr(T > t)$. Thus,

$$\begin{aligned} h(t) &= \lim_{\Delta t \rightarrow 0} \frac{\Pr((t < T \leq t + \Delta t) \cap (T > t)) / \Delta t}{\Pr(T > t)} \\ &= \lim_{\Delta t \rightarrow 0} \frac{\Pr(t < T \leq t + \Delta t) / \Delta t}{\Pr(T > t)} \\ &= \frac{f(t)}{S(t)}, \end{aligned} \tag{11.10}$$

where

$$f(t) = \lim_{\Delta t \rightarrow 0} \frac{\Pr(t < T \leq t + \Delta t)}{\Delta t} \tag{11.11}$$

is the *probability density function* associated with T , i.e. it is the instantaneous rate of death at time t . The second equality in (11.10) made use of the fact that if $t < T \leq t + \Delta t$, then it must be the case that $T > t$.

probability
density
function

Equation 11.10 implies a relationship between the hazard function $h(t)$, the survival function $S(t)$, and the probability density function $f(t)$. In fact, these are three equivalent ways⁸ of describing the distribution of T .

The likelihood associated with the i th observation is

$$\begin{aligned} L_i &= \begin{cases} f(y_i) & \text{if the } i\text{th observation is not censored} \\ S(y_i) & \text{if the } i\text{th observation is censored} \end{cases} \\ &= f(y_i)^{\delta_i} S(y_i)^{1-\delta_i}. \end{aligned} \tag{11.12}$$

The intuition behind (11.12) is as follows: if $Y = y_i$ and the i th observation is not censored, then the likelihood is the probability of dying in a tiny interval around time y_i . If the i th observation is censored, then the likelihood

⁷Due to the Δt in the denominator of (11.9), the hazard function is a rate of death, rather than a probability of death. However, higher values of $h(t)$ directly correspond to a higher probability of death, just as higher values of a probability density function correspond to more likely outcomes for a random variable. In fact, $h(t)$ is the probability density function for T conditional on $T > t$.

⁸See Exercise 8.

is the probability of surviving at least until time y_i . Assuming that the n observations are independent, the likelihood for the data takes the form

$$L = \prod_{i=1}^n f(y_i)^{\delta_i} S(y_i)^{1-\delta_i} = \prod_{i=1}^n h(y_i)^{\delta_i} S(y_i), \quad (11.13)$$

where the second equality follows from (11.10).

We now consider the task of modeling the survival times. If we assume exponential survival, i.e. that the probability density function of the survival time T takes the form $f(t) = \lambda \exp(-\lambda t)$, then estimating the parameter λ by maximizing the likelihood in (11.13) is straightforward.⁹ Alternatively, we could assume that the survival times are drawn from a more flexible family of distributions, such as the Gamma or Weibull family. Another possibility is to model the survival times non-parametrically, as was done in Section 11.3 using the Kaplan–Meier estimator.

However, what we would really like to do is model the survival time *as a function of the covariates*. To do this, it is convenient to work directly with the hazard function, instead of the probability density function.¹⁰ One possible approach is to assume a functional form for the hazard function $h(t|x_i)$, such as $h(t|x_i) = \exp\left(\beta_0 + \sum_{j=1}^p \beta_j x_{ij}\right)$, where the exponent function guarantees that the hazard function is non-negative. Note that the exponential hazard function is special, in that it does not vary with time.¹¹ Given $h(t|x_i)$, we could calculate $S(t|x_i)$. Plugging these equations into (11.13), we could then maximize the likelihood in order to estimate the parameter $\beta = (\beta_0, \beta_1, \dots, \beta_p)^T$. However, this approach is quite restrictive, in the sense that it requires us to make a very stringent assumption on the form of the hazard function $h(t|x_i)$. In the next section, we will consider a much more flexible approach.

11.5.2 Proportional Hazards

The Proportional Hazards Assumption

The *proportional hazards assumption* states that

$$h(t|x_i) = h_0(t) \exp\left(\sum_{j=1}^p x_{ij} \beta_j\right), \quad (11.14)$$

proportional
hazards
assumption

where $h_0(t) \geq 0$ is an unspecified function, known as the *baseline hazard*. It is the hazard function for an individual with features $x_{i1} = \dots = x_{ip} = 0$. The name “proportional hazards” arises from the fact that the hazard function for an individual with feature vector x_i is some unknown function

baseline
hazard

⁹See Exercise 9.

¹⁰Given the close relationship between the hazard function $h(t)$ and the density function $f(t)$ explored in Exercise 8, posing an assumption about the form of the hazard function is closely related to posing an assumption about the form of the density function, as was done in the previous paragraph.

¹¹The notation $h(t|x_i)$ indicates that we are now considering the hazard function for the i th observation conditional on the values of the covariates, x_i .

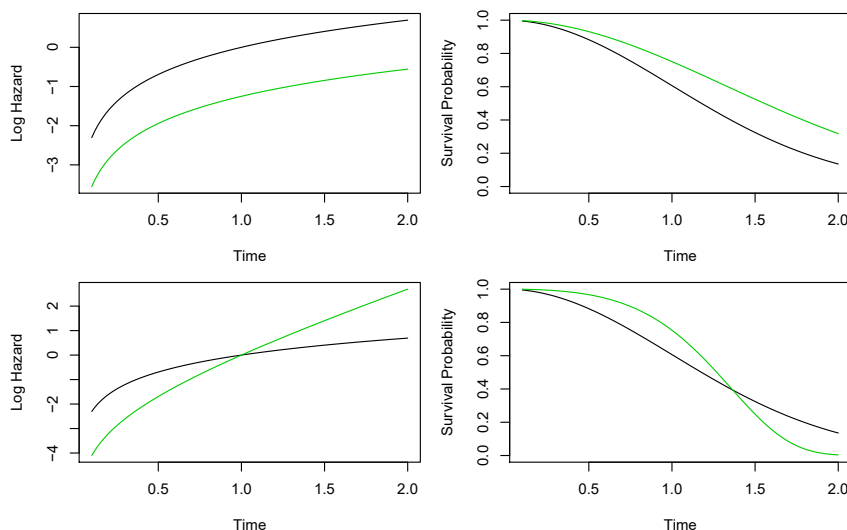


FIGURE 11.4. Top: In a simple example with $p = 1$ and a binary covariate $x_i \in \{0, 1\}$, the log hazard and the survival function under the model (11.14) are shown (green for $x_i = 0$ and black for $x_i = 1$). Because of the proportional hazards assumption (11.14), the log hazard functions differ by a constant, and the survival functions do not cross. Bottom: Again we have a single binary covariate $x_i \in \{0, 1\}$. However, the proportional hazards assumption (11.14) does not hold. The log hazard functions cross, as do the survival functions.

$h_0(t)$ times the factor $\exp\left(\sum_{j=1}^p x_{ij}\beta_j\right)$. The quantity $\exp\left(\sum_{j=1}^p x_{ij}\beta_j\right)$ is called the *relative risk* for the feature vector $x_i = (x_{i1}, \dots, x_{ip})^T$, relative to that for the feature vector $x_i = (0, \dots, 0)^T$.

What does it mean that the baseline hazard function $h_0(t)$ in (11.14) is unspecified? Basically, we make no assumptions about its functional form. We allow the instantaneous probability of death at time t , given that one has survived at least until time t , to take any form. This means that the hazard function is very flexible and can model a wide range of relationships between the covariates and survival time. Our only assumption is that a one-unit increase in x_{ij} corresponds to an increase in $h(t|x_i)$ by a factor of $\exp(\beta_j)$.

An illustration of the proportional hazards assumption (11.14) is given in Figure 11.4, in a simple setting with a single binary covariate $x_i \in \{0, 1\}$ (so that $p = 1$). In the top row, the proportional hazards assumption (11.14) holds. Thus, the hazard functions of the two groups are a constant multiple of each other, so that on the log scale, the gap between them is constant. Furthermore, the survival curves never cross, and in fact the gap between the survival curves tends to (initially) increase over time. By contrast, in the bottom row, (11.14) does not hold. We see that the log hazard functions for the two groups cross, as do the survival curves.

Cox's Proportional Hazards Model

Because the form of $h_0(t)$ in the proportional hazards assumption (11.14) is unknown, we cannot simply plug $h(t|x_i)$ into the likelihood (11.13) and then estimate $\beta = (\beta_1, \dots, \beta_p)^T$ by maximum likelihood. The magic of *Cox's proportional hazards model* lies in the fact that it is in fact possible to estimate β *without having to specify the form of* $h_0(t)$.

Cox's
proportional
hazards
model

To accomplish this, we make use of the same “sequential in time” logic that we used to derive the Kaplan–Meier survival curve and the log-rank test. For simplicity, assume that there are no ties among the failure, or death, times: i.e. each failure occurs at a distinct time. Assume that $\delta_i = 1$, i.e. the i th observation is uncensored, and thus y_i is its failure time. Then the hazard function for the i th observation at time y_i is $h(y_i|x_i) = h_0(y_i) \exp\left(\sum_{j=1}^p x_{ij}\beta_j\right)$, and the total hazard at time y_i for the at risk observations¹² is

$$\sum_{i': y_{i'} \geq y_i} h_0(y_i) \exp\left(\sum_{j=1}^p x_{i'j}\beta_j\right).$$

Therefore, the probability that the i th observation is the one to fail at time y_i (as opposed to one of the other observations in the risk set) is

$$\frac{h_0(y_i) \exp\left(\sum_{j=1}^p x_{ij}\beta_j\right)}{\sum_{i': y_{i'} \geq y_i} h_0(y_i) \exp\left(\sum_{j=1}^p x_{i'j}\beta_j\right)} = \frac{\exp\left(\sum_{j=1}^p x_{ij}\beta_j\right)}{\sum_{i': y_{i'} \geq y_i} \exp\left(\sum_{j=1}^p x_{i'j}\beta_j\right)}. \quad (11.15)$$

Notice that the unspecified baseline hazard function $h_0(y_i)$ cancels out of the numerator and denominator!

The *partial likelihood* is simply the product of these probabilities over all of the uncensored observations,

partial
likelihood

$$PL(\beta) = \prod_{i: \delta_i=1} \frac{\exp\left(\sum_{j=1}^p x_{ij}\beta_j\right)}{\sum_{i': y_{i'} \geq y_i} \exp\left(\sum_{j=1}^p x_{i'j}\beta_j\right)}. \quad (11.16)$$

Critically, the partial likelihood is valid regardless of the true value of $h_0(t)$, making the model very flexible and robust.¹³

To estimate β , we simply maximize the partial likelihood (11.16) with respect to β . As was the case for logistic regression in Chapter 4, no closed-form solution is available, and so iterative algorithms are required.

In addition to estimating β , we can also obtain other model outputs that we saw in the context of least squares regression in Chapter 3 and logistic regression in Chapter 4. For example, we can obtain p -values corresponding

¹²Recall that the “at risk” observations at time y_i are those that are still at risk of failure, i.e. those that have not yet failed or been censored before time y_i .

¹³In general, the partial likelihood is used in settings where it is difficult to compute the full likelihood for all of the parameters. Instead, we compute a likelihood for just the parameters of primary interest: in this case, $\beta_1, \dots, \beta_p$. It can be shown that maximizing (11.16) provides good estimates for these parameters.

to particular null hypotheses (e.g. $H_0 : \beta_j = 0$), as well as confidence intervals associated with the coefficients.

Connection With The Log-Rank Test

Suppose we have just a single predictor ($p = 1$), which we assume to be binary, i.e. $x_i \in \{0, 1\}$. In order to determine whether there is a difference between the survival times of the observations in the group $\{i : x_i = 0\}$ and those in the group $\{i : x_i = 1\}$, we can consider taking two possible approaches:

Approach #1: Fit a Cox proportional hazards model, and test the null hypothesis $H_0 : \beta = 0$. (Since $p = 1$, β is a scalar.)

Approach #2: Perform a log-rank test to compare the two groups, as in Section 11.4.

Which one should we prefer?

In fact, there is a close relationship between these two approaches. In particular, when taking Approach #1, there are a number of possible ways to test H_0 . One way is known as a score test. It turns out that in the case of a single binary covariate, the score test for $H_0 : \beta = 0$ in Cox's proportional hazards model is exactly equal to the log-rank test. In other words, it does not matter whether we take Approach #1 or Approach #2!

Additional Details

The discussion of Cox's proportional hazards model glossed over a few subtleties:

- There is no intercept in (11.14) nor in the equations that follow, because an intercept can be absorbed into the baseline hazard $h_0(t)$.
- We have assumed that there are no tied failure times. In the case of ties, the exact form of the partial likelihood (11.16) is a bit more complicated, and a number of computational approximations must be used.
- (11.16) is known as the *partial* likelihood because it is not exactly a likelihood. That is, it does not correspond exactly to the probability of the data under the assumption (11.14). However, it is a very good approximation.
- We have focused only on estimation of the coefficients $\beta = (\beta_1, \dots, \beta_p)^T$. However, at times we may also wish to estimate the baseline hazard $h_0(t)$, for instance so that we can estimate the survival curve $S(t|x)$ for an individual with feature vector x . The details are beyond the scope of this book. Estimation of $h_0(t)$ is implemented in the **lifelines** package in **Python**, which we will see in Section 11.8.

11.5.3 Example: Brain Cancer Data

Table 11.2 shows the result of fitting the proportional hazards model to the **BrainCancer** data, which was originally described in Section 11.3. The coefficient column displays $\hat{\beta}_j$. The results indicate, for instance, that the estimated hazard for a male patient is $e^{0.18} = 1.2$ times greater than for a female patient: in other words, with all other features held fixed, males have a 1.2 times greater chance of dying than females, at any point in time. However, the p -value is 0.61, which indicates that this difference between males and females is not significant.

As another example, we also see that each one-unit increase in the Karnofsky index corresponds to a multiplier of $\exp(-0.05) = 0.95$ in the instantaneous chance of dying. In other words, the higher the Karnofsky index, the lower the chance of dying at any given point in time. This effect is highly significant, with a p -value of 0.0027.

	Coefficient	Std. error	z-statistic	p-value
sex [Male]	0.18	0.36	0.51	0.61
diagnosis [LG Glioma]	0.92	0.64	1.43	0.15
diagnosis [HG Glioma]	2.15	0.45	4.78	0.00
diagnosis [Other]	0.89	0.66	1.35	0.18
loc [Supratentorial]	0.44	0.70	0.63	0.53
ki	-0.05	0.02	-3.00	<0.01
gtv	0.03	0.02	1.54	0.12
stereo [SRT]	0.18	0.60	0.30	0.77

TABLE 11.2. Results for Cox’s proportional hazards model fit to the **BrainCancer** data, which was first described in Section 11.3. The variable **diagnosis** is qualitative with four levels: meningioma, LG glioma, HG glioma, or other. The variables **sex**, **loc**, and **stereo** are binary.

11.5.4 Example: Publication Data

Next, we consider the dataset **Publication** involving the time to publication of journal papers reporting the results of clinical trials funded by the National Heart, Lung, and Blood Institute.¹⁴ For 244 trials, the time in months until publication is recorded. Of the 244 trials, only 156 were published during the study period; the remaining studies were censored. The covariates include whether the trial focused on a clinical endpoint (**clinend**), whether the trial involved multiple centers (**multi**), the funding mechanism within the National Institutes of Health (**mech**), trial sample size (**sampsize**), budget (**budget**), impact (**impact**, related to the number of citations), and whether the trial produced a positive (significant) result (**posres**). The last covariate is particularly interesting, as a number of studies have suggested that positive trials have a higher publication rate.

¹⁴This dataset is described in the following paper: Gordon et al. (2013) Publication of trials funded by the National Heart, Lung, and Blood Institute. *New England Journal of Medicine*, 369(20):1926–1934.

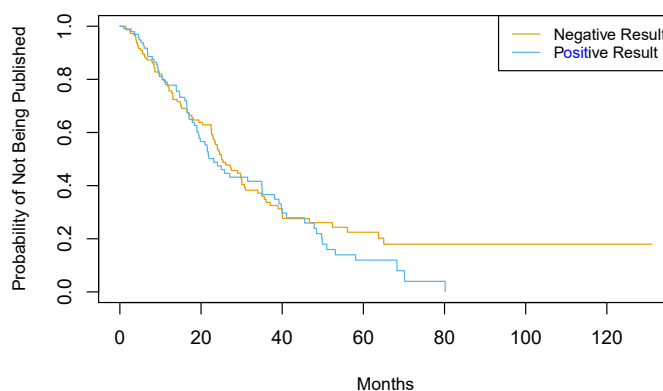


FIGURE 11.5. Survival curves for time until publication for the **Publication** data described in Section 11.5.4, stratified by whether or not the study produced a positive result.

Figure 11.5 shows the Kaplan–Meier curves for the time until publication, stratified by whether or not the study produced a positive result. We see slight evidence that time until publication is lower for studies with a positive result. However, the log-rank test yields a very unimpressive p -value of 0.36.

We now consider a more careful analysis that makes use of all of the available predictors. The results of fitting Cox’s proportional hazards model using all of the available features are shown in Table 11.3. We find that the chance of publication of a study with a positive result is $e^{0.55} = 1.74$ times higher than the chance of publication of a study with a negative result at any point in time, holding all other covariates fixed. The very small p -value associated with **posres** in Table 11.3 indicates that this result is highly significant. This is striking, especially in light of our earlier finding that a log-rank test comparing time to publication for studies with positive versus negative results yielded a p -value of 0.36. How can we explain this discrepancy? The answer stems from the fact that the log-rank test did not consider any other covariates, whereas the results in Table 11.3 are based on a Cox model using all of the available covariates. In other words, after we adjust for all of the other covariates, then whether or not the study yielded a positive result is highly predictive of the time to publication.

In order to gain more insight into this result, in Figure 11.6 we display estimates of the survival curves associated with positive and negative results, adjusting for the other predictors. To produce these survival curves, we estimated the underlying baseline hazard $h_0(t)$. We also needed to select representative values for the other predictors; we used the mean value for each predictor, except for the categorical predictor **mech**, for which we used the most prevalent category (**R01**). Adjusting for the other predictors, we now see a clear difference in the survival curves between studies with positive versus negative results.

Other interesting insights can be gleaned from Table 11.3. For example, studies with a clinical endpoint are more likely to be published at any given point in time than those with a non-clinical endpoint. The funding

	Coefficient	Std. error	<i>z</i> -statistic	<i>p</i> -value
posres[Yes]	0.55	0.18	3.02	0.00
multi[Yes]	0.15	0.31	0.47	0.64
clinend[Yes]	0.51	0.27	1.89	0.06
mech[K01]	1.05	1.06	1.00	0.32
mech[K23]	-0.48	1.05	-0.45	0.65
mech[P01]	-0.31	0.78	-0.40	0.69
mech[P50]	0.60	1.06	0.57	0.57
mech[R01]	0.10	0.32	0.30	0.76
mech[R18]	1.05	1.05	0.99	0.32
mech[R21]	-0.05	1.06	-0.04	0.97
mech[R24,K24]	0.81	1.05	0.77	0.44
mech[R42]	-14.78	3414.38	-0.00	1.00
mech[R44]	-0.57	0.77	-0.73	0.46
mech[RC2]	-14.92	2243.60	-0.01	0.99
mech[U01]	-0.22	0.32	-0.70	0.48
mech[U54]	0.47	1.07	0.44	0.66
sampsize	0.00	0.00	0.19	0.85
budget	0.00	0.00	1.67	0.09
impact	0.06	0.01	8.23	0.00

TABLE 11.3. Results for Cox’s proportional hazards model fit to the **Publication** data, using all of the available features. The features **posres**, **multi**, and **clinend** are binary. The feature **mech** is qualitative with 14 levels; it is coded so that the baseline level is **Contract**.

mechanism did not appear to be significantly associated with time until publication.

11.6 Shrinkage for the Cox Model

In this section, we illustrate that the shrinkage methods of Section 6.2 can be applied to the survival data setting. In particular, motivated by the “loss+penalty” formulation of Section 6.2, we consider minimizing a penalized version of the negative log partial likelihood in (11.16),

$$-\log \left(\prod_{i:\delta_i=1} \frac{\exp \left(\sum_{j=1}^p x_{ij} \beta_j \right)}{\sum_{i': y_{i'} \geq y_i} \exp \left(\sum_{j=1}^p x_{i'j} \beta_j \right)} \right) + \lambda P(\beta), \quad (11.17)$$

with respect to $\beta = (\beta_1, \dots, \beta_p)^T$. We might take $P(\beta) = \sum_{j=1}^p \beta_j^2$, which corresponds to a ridge penalty, or $P(\beta) = \sum_{j=1}^p |\beta_j|$, which corresponds to a lasso penalty.

In (11.17), λ is a non-negative tuning parameter; typically we will minimize it over a range of values of λ . When $\lambda = 0$, then minimizing (11.17) is equivalent to simply maximizing the usual Cox partial likelihood (11.16). However, when $\lambda > 0$, then minimizing (11.17) yields a shrunken version of the coefficient estimates. When λ is large, then using a ridge penalty will give small coefficients that are not exactly equal to zero. By contrast, for a

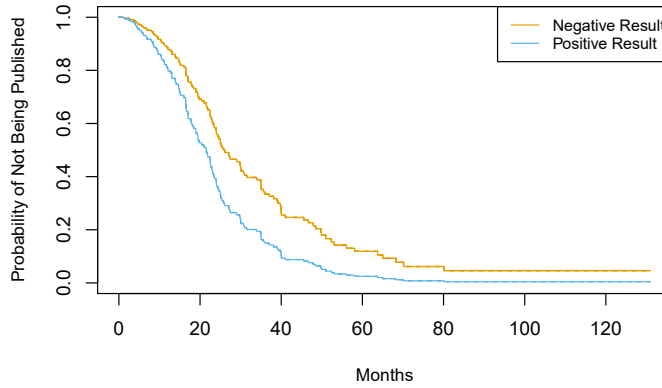


FIGURE 11.6. For the **Publication** data, we display survival curves for time until publication, stratified by whether or not the study produced a positive result, after adjusting for all other covariates.

sufficiently large value of λ , using a lasso penalty will give some coefficients that are exactly equal to zero.

We now apply the lasso-penalized Cox model to the **Publication** data, described in Section 11.5.4. We first randomly split the 244 trials into equally-sized training and test sets. The cross-validation results from the training set are shown in Figure 11.7. The “partial likelihood deviance”, shown on the y -axis, is twice the cross-validated negative log partial likelihood; it plays the role of the cross-validation error.¹⁵ Note the “U-shape” of the partial likelihood deviance: just as we saw in previous chapters, the cross-validation error is minimized for an intermediate level of model complexity. Specifically, this occurs when just two predictors, **budget** and **impact**, have non-zero estimated coefficients.

Now, how do we apply this model to the test set? This brings up an important conceptual point: in essence, there is no simple way to compare predicted survival times and true survival times on the test set. The first problem is that some of the observations are censored, and so the true survival times for those observations are unobserved. The second issue arises from the fact that in the Cox model, rather than predicting a single survival time given a covariate vector x , we instead estimate an entire survival curve, $S(t|x)$, as a function of t .

Therefore, to assess the model fit, we must take a different approach, which involves stratifying the observations using the coefficient estimates. In particular, for each test observation, we compute the “risk” score

$$\text{budget}_i \cdot \hat{\beta}_{\text{budget}} + \text{impact}_i \cdot \hat{\beta}_{\text{impact}},$$

where $\hat{\beta}_{\text{budget}}$ and $\hat{\beta}_{\text{impact}}$ are the coefficient estimates for these two features from the training set. We then use these risk scores to categorize the observations based on their “risk”. For instance, the high risk group consists of the observations for which $\text{budget}_i \cdot \hat{\beta}_{\text{budget}} + \text{impact}_i \cdot \hat{\beta}_{\text{impact}}$ is largest; by

¹⁵Cross-validation for the Cox model is more involved than for linear or logistic regression, because the objective function is not a sum over the observations.

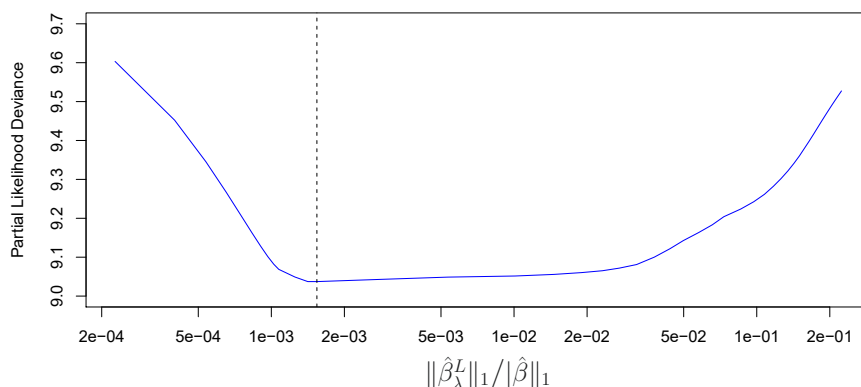


FIGURE 11.7. For the **Publication** data described in Section 11.5.4, cross-validation results for the lasso-penalized Cox model are shown. The y-axis displays the partial likelihood deviance, which plays the role of the cross-validation error. The x-axis displays the ℓ_1 norm (that is, the sum of the absolute values) of the coefficients of the lasso-penalized Cox model with tuning parameter λ , divided by the ℓ_1 norm of the coefficients of the unpenalized Cox model. The dashed line indicates the minimum cross-validation error.

(11.14), we see that these are the observations for which the instantaneous probability of being published at any moment in time is largest. In other words, the high risk group consists of the trials that are likely to be published sooner. On the **Publication** data, we stratify the observations into tertiles of low, medium, and high risk. The resulting survival curves for each of the three strata are displayed in Figure 11.8. We see that there is clear separation between the three strata, and that the strata are correctly ordered in terms of low, medium, and high risk of publication.

11.7 Additional Topics

11.7.1 Area Under the Curve for Survival Analysis

In Chapter 4, we introduced the area under the ROC curve — often referred to as the “AUC” — as a way to quantify the performance of a two-class classifier. Define the *score* for the i th observation to be the classifier’s estimate of $\Pr(Y = 1|X = x_i)$. It turns out that if we consider all pairs consisting of one observation in Class 1 and one observation in Class 2, then the AUC is the fraction of pairs for which the score for the observation in Class 1 exceeds the score for the observation in Class 2.

This suggests a way to generalize the notion of AUC to survival analysis. We calculate an estimated risk score, $\hat{\eta}_i = \hat{\beta}_1 x_{i1} + \cdots + \hat{\beta}_p x_{ip}$, for $i = 1, \dots, n$, using the Cox model coefficients. If $\hat{\eta}_{i'} > \hat{\eta}_i$, then the model predicts that the i' th observation has a larger hazard than the i th observation, and thus that the survival time t_i will be *greater* than $t_{i'}$. Thus, it is tempting to try to generalize AUC by computing the proportion of observations for which $t_i > t_{i'}$ and $\hat{\eta}_i > \hat{\eta}_{i'}$. However, things are not quite so easy, because recall that we do not observe $t_1, \dots, t_n$; instead, we observe

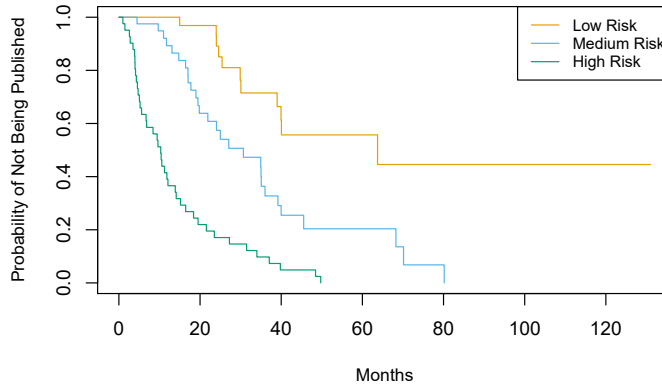


FIGURE 11.8. For the **Publication** data introduced in Section 11.5.4, we compute tertiles of “risk” in the test set using coefficients estimated on the training set. There is clear separation between the resulting survival curves.

the (possibly-censored) times $y_1, \dots, y_n$, as well as the censoring indicators $\delta_1, \dots, \delta_n$.

Therefore, *Harrell’s concordance index* (or *C-index*) computes the proportion of observation pairs for which $\hat{\eta}_{i'} > \hat{\eta}_i$ and $y_i > y_{i'}$:

Harrell’s
concordance
index

$$C = \frac{\sum_{i,i': y_i > y_{i'}} I(\hat{\eta}_{i'} > \hat{\eta}_i) \delta_{i'}}{\sum_{i,i': y_i > y_{i'}} \delta_{i'}},$$

where the indicator variable $I(\hat{\eta}_{i'} > \hat{\eta}_i)$ equals one if $\hat{\eta}_{i'} > \hat{\eta}_i$, and equals zero otherwise. The numerator and denominator are multiplied by the status indicator $\delta_{i'}$, since if the i' th observation is uncensored (i.e. if $\delta_{i'} = 1$), then $y_i > y_{i'}$ implies that $t_i > t_{i'}$. By contrast, if $\delta_{i'} = 0$, then $y_i > y_{i'}$ does not imply that $t_i > t_{i'}$.

We fit a Cox proportional hazards model on the training set of the **Publication** data, and computed the *C-index* on the test set. This yielded $C = 0.733$. Roughly speaking, given two random papers from the test set, the model can predict with 73.3% accuracy which will be published first.

11.7.2 Choice of Time Scale

In the examples considered thus far in this chapter, it has been fairly clear how to define *time*. For example, in the **Publication** example, *time zero* for each paper was defined to be the calendar time at the end of the study, and the failure time was defined to be the number of months that elapsed from the end of the study until the paper was published.

However, in other settings, the definitions of time zero and failure time may be more subtle. For example, when examining the association between risk factors and disease occurrence in an epidemiological study, one might use the patient’s age to define time, so that time zero is the patient’s date of birth. With this choice, the association between age and survival cannot be measured; however, there is no need to adjust for age in the analysis. When examining covariates associated with disease-free survival (i.e. the

amount of time elapsed between treatment and disease recurrence), one might use the date of treatment as time zero.

11.7.3 Time-Dependent Covariates

A powerful feature of the proportional hazards model is its ability to handle *time-dependent covariates*, predictors whose value may change over time. For example, suppose we measure a patient's blood pressure every week over the course of a medical study. In this case, we can think of the blood pressure for the i th observation not as x_i , but rather as $x_i(t)$ at time t .

Because the partial likelihood in (11.16) is constructed sequentially in time, dealing with time-dependent covariates is straightforward. In particular, we simply replace x_{ij} and $x_{i'j}$ in (11.16) with $x_{ij}(y_i)$ and $x_{i'j}(y_i)$, respectively; these are the current values of the predictors at time y_i . By contrast, time-dependent covariates would pose a much greater challenge within the context of a traditional parametric approach, such as (11.13).

One example of time-dependent covariates appears in the analysis of data from the Stanford Heart Transplant Program. Patients in need of a heart transplant were put on a waiting list. Some patients received a transplant, but others died while still on the waiting list. The primary objective of the analysis was to determine whether a transplant was associated with longer patient survival.

A naïve approach would use a fixed covariate to represent transplant status: that is, $x_i = 1$ if the i th patient ever received a transplant, and $x_i = 0$ otherwise. But this approach overlooks the fact that patients had to live long enough to get a transplant, and hence, on average, healthier patients received transplants. This problem can be solved by using a time-dependent covariate for transplant: $x_i(t) = 1$ if the patient received a transplant by time t , and $x_i(t) = 0$ otherwise.

11.7.4 Checking the Proportional Hazards Assumption

We have seen that Cox's proportional hazards model relies on the proportional hazards assumption (11.14). While results from the Cox model tend to be fairly robust to violations of this assumption, it is still a good idea to check whether it holds. In the case of a qualitative feature, we can plot the log hazard function for each level of the feature. If (11.14) holds, then the log hazard functions should just differ by a constant, as seen in the top-left panel of Figure 11.4. In the case of a quantitative feature, we can take a similar approach by stratifying the feature.

11.7.5 Survival Trees

In Chapter 8, we discussed flexible and adaptive learning procedures such as trees, random forests, and boosting, which we applied in both the regression and classification settings. Most of these approaches can be generalized to the survival analysis setting. For example, *survival trees* are a modification of classification and regression trees that use a split criterion that maximizes

survival
trees

the difference between the survival curves in the resulting daughter nodes. Survival trees can then be used to create random survival forests.

11.8 Lab: Survival Analysis

In this lab, we perform survival analyses on three separate data sets. In Section 11.8.1 we analyze the `BrainCancer` data that was first described in Section 11.3. In Section 11.8.2, we examine the `Publication` data from Section 11.5.4. Finally, Section 11.8.3 explores a simulated call-center data set.

We begin by importing some of our libraries at this top level. This makes the code more readable, as scanning the first few lines of the notebook tell us what libraries are used in this notebook.

```
In [1]: from matplotlib.pyplot import subplots
import numpy as np
import pandas as pd
from ISLP.models import ModelSpec as MS
from ISLP import load_data
```

We also collect the new imports needed for this lab.

```
In [2]: from lifelines import \
        (KaplanMeierFitter,
         CoxPHFitter)
from lifelines.statistics import \
        (logrank_test,
         multivariate_logrank_test)
from ISLP.survival import sim_time
```

11.8.1 Brain Cancer Data

We begin with the `BrainCancer` data set, contained in the `ISLP` package.

```
In [3]: BrainCancer = load_data('BrainCancer')
BrainCancer.columns
```

```
Out[3]: Index(['sex', 'diagnosis', 'loc', 'ki', 'gtv', 'stereo',
              'status', 'time'],
              dtype='object')
```

The rows index the 88 patients, while the 8 columns contain the predictors and outcome variables. We first briefly examine the data.

```
In [4]: BrainCancer['sex'].value_counts()
```

```
Out[4]: Female    45
        Male      43
        Name: sex, dtype: int64
```

```
In [5]: BrainCancer['diagnosis'].value_counts()
```

```
Out[5]: Meningioma      42
        HG glioma      22
        Other          14
        LG glioma       9
        Name: diagnosis, dtype: int64
```

```
In [6]: BrainCancer['status'].value_counts()
```

```
Out[6]: 0      53
        1      35
        Name: status, dtype: int64
```

Before beginning an analysis, it is important to know how the `status` variable has been coded. Most software uses the convention that a `status` of 1 indicates an uncensored observation (often death), and a `status` of 0 indicates a censored observation. But some scientists might use the opposite coding. For the `BrainCancer` data set 35 patients died before the end of the study, so we are using the conventional coding.

To begin the analysis, we re-create the Kaplan-Meier survival curve shown in Figure 11.2. The main package we will use for survival analysis is `lifelines`. The variable `time` corresponds to y_i , the time to the i th event (either censoring or death). The first argument to `km.fit` is the event time, and the second argument is the censoring variable, with a 1 indicating an observed failure time. The `plot()` method produces a survival curve with pointwise confidence intervals. By default, these are 90% confidence intervals, but this can be changed by setting the `alpha` argument to one minus the desired confidence level.

`lifelines`

`.plot()`

```
In [7]: fig, ax = subplots(figsize=(8,8))
        km = KaplanMeierFitter()
        km_brain = km.fit(BrainCancer['time'], BrainCancer['status'])
        km_brain.plot(label='Kaplan Meier estimate', ax=ax)
```

Next we create Kaplan-Meier survival curves that are stratified by `sex`, in order to reproduce Figure 11.3. We do this using the `groupby()` method of a dataframe. This method returns a generator that can be iterated over in the `for` loop. In this case, the items in the `for` loop are 2-tuples representing the groups: the first entry is the value of the grouping column `sex` while the second value is the dataframe consisting of all rows in the dataframe matching that value of `sex`. We will want to use this data below in the log-rank test, hence we store this information in the dictionary `by_sex`. Finally, we have also used the notion of *string interpolation* to automatically label the different lines in the plot. String interpolation is a powerful technique to format strings — `Python` has many ways to facilitate such operations.

`.groupby()`

string
interpolation

```
In [8]: fig, ax = subplots(figsize=(8,8))
        by_sex = {}
        for sex, df in BrainCancer.groupby('sex'):
            by_sex[sex] = df
            km_sex = km.fit(df['time'], df['status'])
            km_sex.plot(label='Sex=%s' % sex, ax=ax)
```

As discussed in Section 11.4, we can perform a log-rank test to compare the survival of males to females. We use the `logrank_test()` function from

`logrank_`
`test()`

the `lifelines.statistics` module. The first two arguments are the event times, with the second denoting the corresponding (optional) censoring indicators.

```
In [9]: logrank_test(by_sex['Male']['time'],
                    by_sex['Female']['time'],
                    by_sex['Male']['status'],
                    by_sex['Female']['status'])
```

```
Out[9]:
```

	t_0	-1
null_distribution	chi	squared
degrees_of_freedom		1
test_name	logrank_test	

test_statistic	p	-log2(p)
1.44	0.23	2.12

The resulting p -value is 0.23, indicating no evidence of a difference in survival between the two sexes.

Next, we use the `CoxPHFitter()` estimator from `lifelines` to fit Cox proportional hazards models. To begin, we consider a model that uses `sex` as the only predictor. `CoxPHFitter()`

```
In [10]: coxph = CoxPHFitter # shorthand
sex_df = BrainCancer[['time', 'status', 'sex']]
model_df = MS(['time', 'status', 'sex'],
              intercept=False).fit_transform(sex_df)
cox_fit = coxph().fit(model_df,
                    'time',
                    'status')
cox_fit.summary[['coef', 'se(coef)', 'p']]
```

```
Out[10]:
```

	coef	se(coef)	p
covariate			
sex[Male]	0.407667	0.342004	0.233263

The first argument to `fit` should be a data frame containing at least the event time (the second argument `time` in this case), as well as an optional censoring variable (the argument `status` in this case). Note also that the Cox model does not include an intercept, which is why we used the `intercept=False` argument to `ModelSpec` above. The `summary()` method delivers many columns; we chose to abbreviate its output here. It is possible to obtain the likelihood ratio test comparing this model to the one with no features as follows:

```
In [11]: cox_fit.log_likelihood_ratio_test()
```

```
Out[11]:
```

null_distribution		chi	squared
degrees_freedom			1
test_name	log-likelihood	ratio	test

test_statistic	p	-log2(p)
1.44	0.23	2.12

Regardless of which test we use, we see that there is no clear evidence for a difference in survival between males and females. As we learned in this chapter, the score test from the Cox model is exactly equal to the log rank test statistic!

Now we fit a model that makes use of additional predictors. We first note that one of our `diagnosis` values is missing, hence we drop that observation before continuing.

```
In [12]: cleaned = BrainCancer.dropna()
all_MS = MS(cleaned.columns, intercept=False)
all_df = all_MS.fit_transform(cleaned)
fit_all = coxph().fit(all_df,
                      'time',
                      'status')
fit_all.summary[['coef', 'se(coef)', 'p']]
```

```
Out[12]:
```

	coef	se(coef)	p
covariate			
sex[Male]	0.183748	0.360358	0.610119
diagnosis[LG glioma]	-1.239541	0.579557	0.032454
diagnosis[Meningioma]	-2.154566	0.450524	0.000002
diagnosis[Other]	-1.268870	0.617672	0.039949
loc[Supratentorial]	0.441195	0.703669	0.530664
ki	-0.054955	0.018314	0.002693
gtv	0.034293	0.022333	0.124660
stereo[SRT]	0.177778	0.601578	0.767597

The `diagnosis` variable has been coded so that the baseline corresponds to HG glioma. The results indicate that the risk associated with HG glioma is more than eight times (i.e. $e^{2.15} = 8.62$) the risk associated with meningioma. In other words, after adjusting for the other predictors, patients with HG glioma have much worse survival compared to those with meningioma. In addition, larger values of the Karnofsky index, `ki`, are associated with lower risk, i.e. longer survival.

Finally, we plot estimated survival curves for each diagnosis category, adjusting for the other predictors. To make these plots, we set the values of the other predictors equal to the mean for quantitative variables and equal to the mode for categorical. To do this, we use the `apply()` method across rows (i.e. `axis=0`) with a function `representative` that checks if a column is categorical or not.

```
In [13]: levels = cleaned['diagnosis'].unique()
def representative(series):
    if hasattr(series.dtype, 'categories'):
        return pd.Series.mode(series)
    else:
        return series.mean()
modal_data = cleaned.apply(representative, axis=0)
```

We make four copies of the column means and assign the `diagnosis` column to be the four different diagnoses.

```
In [14]: modal_df = pd.DataFrame(
    [modal_data.iloc[0] for _ in range(len(levels))]
    modal_df['diagnosis'] = levels
    modal_df
```

```
Out[14]:    sex    diagnosis    loc    ki    gtv    stereo    ...
Female  Meningioma  Supratentorial  80.920  8.687    SRT    ...
Female   HG glioma  Supratentorial  80.920  8.687    SRT    ...
Female   LG glioma  Supratentorial  80.920  8.687    SRT    ...
Female      Other  Supratentorial  80.920  8.687    SRT    ...
```

We then construct the model matrix based on the model specification `all_MS` used to fit the model, and name the rows according to the levels of `diagnosis`.

```
In [15]: modal_X = all_MS.transform(modal_df)
modal_X.index = levels
modal_X
```

We can use the `predict_survival_function()` method to obtain the estimated survival function.

```
In [16]: predicted_survival = fit_all.predict_survival_function(modal_X)
predicted_survival
```

.predict_survival_function()

```
Out[16]:    Meningioma  HG glioma  LG glioma  Other
0.070    0.998    0.982    0.995  0.995
1.180    0.998    0.982    0.995  0.995
1.410    0.996    0.963    0.989  0.990
1.540    0.996    0.963    0.989  0.990
...      ...      ...      ...      ...
67.380    0.689    0.040    0.394  0.405
73.740    0.689    0.040    0.394  0.405
78.750    0.689    0.040    0.394  0.405
82.560    0.689    0.040    0.394  0.405
85 rows x 4 columns
```

This returns a data frame, whose plot methods yields the different survival curves. To avoid clutter in the plots, we do not display confidence intervals.

```
In [17]: fig, ax = subplots(figsize=(8, 8))
predicted_survival.plot(ax=ax);
```

11.8.2 Publication Data

The `Publication` data presented in Section 11.5.4 can be found in the `ISLP` package. We first reproduce Figure 11.5 by plotting the Kaplan-Meier curves stratified on the `posres` variable, which records whether the study had a positive or negative result.

```
In [18]: fig, ax = subplots(figsize=(8,8))
Publication = load_data('Publication')
by_result = {}
for result, df in Publication.groupby('posres'):
    by_result[result] = df
    km_result = km.fit(df['time'], df['status'])
    km_result.plot(label='Result=%d' % result, ax=ax)
```

As discussed previously, the p -values from fitting Cox's proportional hazards model to the `posres` variable are quite large, providing no evidence of a difference in time-to-publication between studies with positive versus negative results.

```
In [19]: posres_df = MS(['posres',
                        'time',
                        'status'],
                        intercept=False).fit_transform(Publication)
posres_fit = coxph().fit(posres_df,
                        'time',
                        'status')
posres_fit.summary[['coef', 'se(coef)', 'p']]
```

```
Out[19]:
```

	coef	se(coef)	p
covariate			
posres	0.148076	0.161625	0.359578

However, the results change dramatically when we include other predictors in the model. Here we exclude the funding mechanism variable.

```
In [20]: model = MS(Publication.columns.drop('mech'),
                    intercept=False)
coxph().fit(model.fit_transform(Publication),
            'time',
            'status').summary[['coef', 'se(coef)', 'p']]
```

```
Out[20]:
```

	coef	se(coef)	p
covariate			
posres	0.570774	0.175960	1.179606e-03
multi	-0.040863	0.251194	8.707727e-01
clinend	0.546180	0.262001	3.710099e-02
sampsize	0.000005	0.000015	7.506978e-01
budget	0.004386	0.002464	7.511276e-02
impact	0.058318	0.006676	2.426779e-18

We see that there are a number of statistically significant variables, including whether the trial focused on a clinical endpoint, the impact of the study, and whether the study had positive or negative results.

11.8.3 Call Center Data

In this section, we will simulate survival data using the relationship between cumulative hazard and the survival function explored in Exercise 8. Our simulated data will represent the observed wait times (in seconds) for 2,000 customers who have phoned a call center. In this context, censoring occurs if a customer hangs up before his or her call is answered.

There are three covariates: **Operators** (the number of call center operators available at the time of the call, which can range from 5 to 15), **Center** (either A, B, or C), and **Time** of day (Morning, Afternoon, or Evening). We generate data for these covariates so that all possibilities are equally likely: for instance, morning, afternoon and evening calls are equally likely, and any number of operators from 5 to 15 is equally likely.

```
In [21]: rng = np.random.default_rng(10)
N = 2000
Operators = rng.choice(np.arange(5, 16),
                       N,
                       replace=True)
```

```
Center = rng.choice(['A', 'B', 'C'],
                    N,
                    replace=True)
Time = rng.choice(['Morn.', 'After.', 'Even.'],
                  N,
                  replace=True)
D = pd.DataFrame({'Operators': Operators,
                  'Center': pd.Categorical(Center),
                  'Time': pd.Categorical(Time)})
```

We then build a model matrix (omitting the intercept)

```
In [22]: model = MS(['Operators',
                    'Center',
                    'Time'],
                    intercept=False)
X = model.fit_transform(D)
```

It is worthwhile to take a peek at the model matrix `X`, so that we can be sure that we understand how the variables have been coded. By default, the levels of categorical variables are sorted and, as usual, the first column of the one-hot encoding of the variable is dropped.

```
In [23]: X[:5]
```

```
Out[23]:   Operators  Center[B]  Center[C]  Time[Even.]  Time[Morn.]
0         13         0.0         1.0         0.0         0.0
1         15         0.0         0.0         1.0         0.0
2          7         1.0         0.0         0.0         1.0
3          7         0.0         1.0         0.0         1.0
4         13         0.0         1.0         1.0         0.0
```

Next, we specify the coefficients and the hazard function.

```
In [24]: true_beta = np.array([0.04, -0.3, 0, 0.2, -0.2])
true_linpred = X.dot(true_beta)
hazard = lambda t: 1e-5 * t
```

Here, we have set the coefficient associated with `Operators` to equal 0.04; in other words, each additional operator leads to a $e^{0.04} = 1.041$ -fold increase in the “risk” that the call will be answered, given the `Center` and `Time` covariates. This makes sense: the greater the number of operators at hand, the shorter the wait time! The coefficient associated with `Center == B` is -0.3 , and `Center == A` is treated as the baseline. This means that the risk of a call being answered at Center B is 0.74 times the risk that it will be answered at Center A; in other words, the wait times are a bit longer at Center B.

Recall from Section 2.3.7 the use of `lambda` for creating short functions on the fly. We use the function `sim_time()` from the `ISLP.survival` package. This function uses the relationship between the survival function and cumulative hazard $S(t) = \exp(-H(t))$ and the specific form of the cumulative hazard function in the Cox model to simulate data based on values of the linear predictor `true_linpred` and the cumulative hazard. We need to provide the cumulative hazard function, which we do here.

```
In [25]: cum_hazard = lambda t: 1e-5 * t**2 / 2
```

`sim_time()`

We are now ready to generate data under the Cox proportional hazards model. We truncate the maximum time to 1000 seconds to keep simulated wait times reasonable. The function `sim_time()` takes a linear predictor, a cumulative hazard function and a random number generator.

```
In [26]: W = np.array([sim_time(l, cum_hazard, rng)
                      for l in true_linpred])
D['Wait time'] = np.clip(W, 0, 1000)
```

We now simulate our censoring variable, for which we assume 90% of calls were answered (`Failed==1`) before the customer hung up (`Failed==0`).

```
In [27]: D['Failed'] = rng.choice([1, 0],
                                N,
                                p=[0.9, 0.1])
D[:5]
```

```
Out[27]:
```

	Operators	Center	Time	Wait time	Failed
0	13	C	After.	525.064979	1
1	15	A	Even.	254.677835	1
2	7	B	Morn.	487.739224	1
3	7	C	Morn.	308.580292	1
4	13	C	Even.	154.174608	1

```
In [28]: D['Failed'].mean()
```

```
Out[28]: 0.8985
```

We now plot Kaplan-Meier survival curves. First, we stratify by `Center`.

```
In [29]: fig, ax = subplots(figsize=(8,8))
by_center = {}
for center, df in D.groupby('Center'):
    by_center[center] = df
    km_center = km.fit(df['Wait time'], df['Failed'])
    km_center.plot(label='Center=%s' % center, ax=ax)
ax.set_title("Probability of Still Being on Hold")
```

Next, we stratify by `Time`.

```
In [30]: fig, ax = subplots(figsize=(8,8))
by_time = {}
for time, df in D.groupby('Time'):
    by_time[time] = df
    km_time = km.fit(df['Wait time'], df['Failed'])
    km_time.plot(label='Time=%s' % time, ax=ax)
ax.set_title("Probability of Still Being on Hold")
```

It seems that calls at Call Center B take longer to be answered than calls at Centers A and C. Similarly, it appears that wait times are longest in the morning and shortest in the evening hours. We can use a log-rank test to determine whether these differences are statistically significant using the function `multivariate_logrank_test()`.

```
In [31]: multivariate_logrank_test(D['Wait time'],
                                D['Center'],
                                D['Failed'])
```

```
Out[31]:          t_0          -1
          null_distribution      chi squared
          degrees_of_freedom      2
          test_name      multivariate_logrank_test

          test_statistic      p      -log2(p)
          20.30 <0.005      14.65
```

Next, we consider the effect of `Time`.

```
In [32]: multivariate_logrank_test(D['Wait time'],
                                   D['Time'],
                                   D['Failed'])
```

```
Out[32]:          t_0          -1
          null_distribution      chi squared
          degrees_of_freedom      2
          test_name      multivariate_logrank_test

          test_statistic      p      -log2(p)
          49.90 <0.005      35.99
```

As in the case of a categorical variable with 2 levels, these results are similar to the likelihood ratio test from the Cox proportional hazards model. First, we look at the results for `Center`.

```
In [33]: X = MS(['Wait time',
                 'Failed',
                 'Center'],
                intercept=False).fit_transform(D)
F = coxph().fit(X, 'Wait time', 'Failed')
F.log_likelihood_ratio_test()
```

```
Out[33]: null_distribution      chi squared
          degrees_freedom      2
          test_name      log-likelihood ratio test

          test_statistic      p      -log2(p)
          20.58 <0.005      14.85
```

Next, we look at the results for `Time`.

```
In [34]: X = MS(['Wait time',
                 'Failed',
                 'Time'],
                intercept=False).fit_transform(D)
F = coxph().fit(X, 'Wait time', 'Failed')
F.log_likelihood_ratio_test()
```

```
Out[34]: null_distribution      chi squared
          degrees_freedom      2
          test_name      log-likelihood ratio test

          test_statistic      p      -log2(p)
          48.12 <0.005      34.71
```

We find that differences between centers are highly significant, as are differences between times of day.

Finally, we fit Cox's proportional hazards model to the data.

```
In [35]: X = MS(D.columns,
              intercept=False).fit_transform(D)
fit_queuing = coxph().fit(
    X,
    'Wait time',
    'Failed')
fit_queuing.summary[['coef', 'se(coef)', 'p']]
```

```
Out[35]:
```

	coef	se(coef)	p
covariate			
Operators	0.043934	0.007520	5.143677e-09
Center[B]	-0.236059	0.058113	4.864734e-05
Center[C]	0.012231	0.057518	8.316083e-01
Time[Even.]	0.268845	0.057797	3.294914e-06
Time[Morn.]	-0.148215	0.057334	9.734378e-03

The p -values for Center B and evening time are very small. It is also clear that the hazard — that is, the instantaneous risk that a call will be answered — increases with the number of operators. Since we generated the data ourselves, we know that the true coefficients for **Operators**, **Center = B**, **Center = C**, **Time = Even.** and **Time = Morn.** are 0.04, -0.3 , 0, 0.2, and -0.2 , respectively. The coefficient estimates from the fitted Cox model are fairly accurate.

11.9 Exercises

Conceptual

- For each example, state whether or not the censoring mechanism is independent. Justify your answer.
 - In a study of disease relapse, due to a careless research scientist, all patients whose phone numbers begin with the number “2” are lost to follow up.
 - In a study of longevity, a formatting error causes all patient ages that exceed 99 years to be lost (i.e. we know that those patients are more than 99 years old, but we do not know their exact ages).
 - Hospital A conducts a study of longevity. However, very sick patients tend to be transferred to Hospital B, and are lost to follow up.
 - In a study of unemployment duration, the people who find work earlier are less motivated to stay in touch with study investigators, and therefore are more likely to be lost to follow up.
 - In a study of pregnancy duration, women who deliver their babies pre-term are more likely to do so away from their usual hospital, and thus are more likely to be censored, relative to women who deliver full-term babies.

- (f) A researcher wishes to model the number of years of education of the residents of a small town. Residents who enroll in college out of town are more likely to be lost to follow up, and are also more likely to attend graduate school, relative to those who attend college in town.
 - (g) Researchers conduct a study of disease-free survival (i.e. time until disease relapse following treatment). Patients who have not relapsed within five years are considered to be cured, and thus their survival time is censored at five years.
 - (h) We wish to model the failure time for some electrical component. This component can be manufactured in Iowa or in Pittsburgh, with no difference in quality. The Iowa factory opened five years ago, and so components manufactured in Iowa are censored at five years. The Pittsburgh factory opened two years ago, so those components are censored at two years.
 - (i) We wish to model the failure time of an electrical component made in two different factories, one of which opened before the other. We have reason to believe that the components manufactured in the factory that opened earlier are of higher quality.
2. We conduct a study with $n = 4$ participants who have just purchased cell phones, in order to model the time until phone replacement. The first participant replaces her phone after 1.2 years. The second participant still has not replaced her phone at the end of the two-year study period. The third participant changes her phone number and is lost to follow up (but has not yet replaced her phone) 1.5 years into the study. The fourth participant replaces her phone after 0.2 years.

For each of the four participants ($i = 1, \dots, 4$), answer the following questions using the notation introduced in Section 11.1:

- (a) Is the participant's cell phone replacement time censored?
 - (b) Is the value of c_i known, and if so, then what is it?
 - (c) Is the value of t_i known, and if so, then what is it?
 - (d) Is the value of y_i known, and if so, then what is it?
 - (e) Is the value of δ_i known, and if so, then what is it?
3. For the example in Exercise 2, report the values of K , $d_1, \dots, d_K$, $r_1, \dots, r_K$, and $q_1, \dots, q_K$, where this notation was defined in Section 11.3.
4. This problem makes use of the Kaplan-Meier survival curve displayed in Figure 11.9. The raw data that went into plotting this survival curve is given in Table 11.4. The covariate column of that table is not needed for this problem.
- (a) What is the estimated probability of survival past 50 days?

Observation (Y)	Censoring Indicator (δ)	Covariate (X)
26.5	1	0.1
37.2	1	11
57.3	1	-0.3
90.8	0	2.8
20.2	0	1.8
89.8	0	0.4

TABLE 11.4. Data used in Exercise 4.

- (b) Write out an analytical expression for the estimated survival function. For instance, your answer might be something along the lines of

$$\hat{S}(t) = \begin{cases} 0.8 & \text{if } t < 31 \\ 0.5 & \text{if } 31 \leq t < 77 \\ 0.22 & \text{if } 77 \leq t. \end{cases}$$

(The previous equation is for illustration only: it is not the correct answer!)

5. Sketch the survival function given by the equation

$$\hat{S}(t) = \begin{cases} 0.8 & \text{if } t < 31 \\ 0.5 & \text{if } 31 \leq t < 77 \\ 0.22 & \text{if } 77 \leq t. \end{cases}$$

Your answer should look something like Figure 11.9.

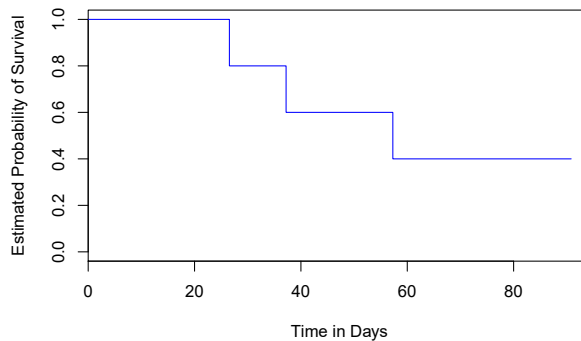


FIGURE 11.9. A Kaplan-Meier survival curve used in Exercise 4.

6. This problem makes use of the data displayed in Figure 11.1. In completing this problem, you can refer to the observation times as $y_1, \dots, y_4$. The ordering of these observation times can be seen from Figure 11.1; their exact values are not required.

- (a) Report the values of $\delta_1, \dots, \delta_4$, K , $d_1, \dots, d_K$, $r_1, \dots, r_K$, and $q_1, \dots, q_K$. The relevant notation is defined in Sections 11.1 and 11.3.

- (b) Sketch the Kaplan-Meier survival curve corresponding to this data set. (You do not need to use any software to do this — you can sketch it by hand using the results obtained in (a).)
- (c) Based on the survival curve estimated in (b), what is the probability that the event occurs within 200 days? What is the probability that the event does not occur within 310 days?
- (d) Write out an expression for the estimated survival curve from (b).
7. In this problem, we will derive (11.5) and (11.6), which are needed for the construction of the log-rank test statistic (11.8). Recall the notation in Table 11.1.



- (a) Assume that there is no difference between the survival functions of the two groups. Then we can think of q_{1k} as the number of failures if we draw r_{1k} observations, without replacement, from a risk set of r_k observations that contains a total of q_k failures. Argue that q_{1k} follows a *hypergeometric distribution*. Write the parameters of this distribution in terms of r_{1k} , r_k , and q_k .
- (b) Given your previous answer, and the properties of the hypergeometric distribution, what are the mean and variance of q_{1k} ? Compare your answer to (11.5) and (11.6).
8. Recall that the survival function $S(t)$, the hazard function $h(t)$, and the density function $f(t)$ are defined in (11.2), (11.9), and (11.11), respectively. Furthermore, define $F(t) = 1 - S(t)$. Show that the following relationships hold:

hyper-
geometric
distribution



$$\begin{aligned} f(t) &= dF(t)/dt \\ S(t) &= \exp\left(-\int_0^t h(u)du\right). \end{aligned}$$

9. In this exercise, we will explore the consequences of assuming that the survival times follow an exponential distribution.
- (a) Suppose that a survival time follows an $\text{Exp}(\lambda)$ distribution, so that its density function is $f(t) = \lambda \exp(-\lambda t)$. Using the relationships provided in Exercise 8, show that $S(t) = \exp(-\lambda t)$.
- (b) Now suppose that each of n independent survival times follows an $\text{Exp}(\lambda)$ distribution. Write out an expression for the likelihood function (11.13).
- (c) Show that the maximum likelihood estimator for λ is

$$\hat{\lambda} = \sum_{i=1}^n \delta_i / \sum_{i=1}^n y_i.$$

- (d) Use your answer to (c) to derive an estimator of the mean survival time.

Hint: For (d), recall that the mean of an $\text{Exp}(\lambda)$ random variable is $1/\lambda$.

Applied

10. This exercise focuses on the brain tumor data, which is included in the **ISLP** library.
 - (a) Plot the Kaplan-Meier survival curve with ± 1 standard error bands, using the `KaplanMeierFitter()` estimator in the **lifelines** package.
 - (b) Draw a bootstrap sample of size $n = 88$ from the pairs (y_i, δ_i) , and compute the resulting Kaplan-Meier survival curve. Repeat this process $B = 200$ times. Use the results to obtain an estimate of the standard error of the Kaplan-Meier survival curve at each timepoint. Compare this to the standard errors obtained in (a).
 - (c) Fit a Cox proportional hazards model that uses all of the predictors to predict survival. Summarize the main findings.
 - (d) Stratify the data by the value of **ki**. (Since only one observation has **ki==40**, you can group that observation together with the observations that have **ki==60**.) Plot Kaplan-Meier survival curves for each of the five strata, adjusted for the other predictors.
11. This exercise makes use of the data in Table 11.4.
 - (a) Create two groups of observations. In Group 1, $X < 2$, whereas in Group 2, $X \geq 2$. Plot the Kaplan-Meier survival curves corresponding to the two groups. Be sure to label the curves so that it is clear which curve corresponds to which group. By eye, does there appear to be a difference between the two groups' survival curves?
 - (b) Fit Cox's proportional hazards model, using the group indicator as a covariate. What is the estimated coefficient? Write a sentence providing the interpretation of this coefficient, in terms of the hazard or the instantaneous probability of the event. Is there evidence that the true coefficient value is non-zero?
 - (c) Recall from Section 11.5.2 that in the case of a single binary covariate, the log-rank test statistic should be identical to the score statistic for the Cox model. Conduct a log-rank test to determine whether there is a difference between the survival curves for the two groups. How does the p -value for the log-rank test statistic compare to the p -value for the score statistic for the Cox model from (b)?



Unsupervised Learning

Most of this book concerns *supervised learning* methods such as regression and classification. In the supervised learning setting, we typically have access to a set of p features $X_1, X_2, \dots, X_p$, measured on n observations, and a response Y also measured on those same n observations. The goal is then to predict Y using $X_1, X_2, \dots, X_p$.

This chapter will instead focus on *unsupervised learning*, a set of statistical tools intended for the setting in which we have only a set of features $X_1, X_2, \dots, X_p$ measured on n observations. We are not interested in prediction, because we do not have an associated response variable Y . Rather, the goal is to discover interesting things about the measurements on $X_1, X_2, \dots, X_p$. Is there an informative way to visualize the data? Can we discover subgroups among the variables or among the observations? Unsupervised learning refers to a diverse set of techniques for answering questions such as these. In this chapter, we will focus on two particular types of unsupervised learning: *principal components analysis*, a tool used for data visualization or data pre-processing before supervised techniques are applied, and *clustering*, a broad class of methods for discovering unknown subgroups in data.

12.1 The Challenge of Unsupervised Learning

Supervised learning is a well-understood area. In fact, if you have read the preceding chapters in this book, then you should by now have a good grasp of supervised learning. For instance, if you are asked to predict a binary outcome from a data set, you have a very well developed set of tools at your disposal (such as logistic regression, linear discriminant analysis, classification trees, support vector machines, and more) as well as a clear

understanding of how to assess the quality of the results obtained (using cross-validation, validation on an independent test set, and so forth).

In contrast, unsupervised learning is often much more challenging. The exercise tends to be more subjective, and there is no simple goal for the analysis, such as prediction of a response. Unsupervised learning is often performed as part of an *exploratory data analysis*. Furthermore, it can be hard to assess the results obtained from unsupervised learning methods, since there is no universally accepted mechanism for performing cross-validation or validating results on an independent data set. The reason for this difference is simple. If we fit a predictive model using a supervised learning technique, then it is possible to *check our work* by seeing how well our model predicts the response Y on observations not used in fitting the model. However, in unsupervised learning, there is no way to check our work because we don't know the true answer—the problem is unsupervised.

exploratory
data
analysis

Techniques for unsupervised learning are of growing importance in a number of fields. A cancer researcher might assay gene expression levels in 100 patients with breast cancer. He or she might then look for subgroups among the breast cancer samples, or among the genes, in order to obtain a better understanding of the disease. An online shopping site might try to identify groups of shoppers with similar browsing and purchase histories, as well as items that are of particular interest to the shoppers within each group. Then an individual shopper can be preferentially shown the items in which he or she is particularly likely to be interested, based on the purchase histories of similar shoppers. A search engine might choose which search results to display to a particular individual based on the click histories of other individuals with similar search patterns. These statistical learning tasks, and many more, can be performed via unsupervised learning techniques.

12.2 Principal Components Analysis

Principal components are discussed in Section 6.3.1 in the context of principal components regression. When faced with a large set of correlated variables, principal components allow us to summarize this set with a smaller number of representative variables that collectively explain most of the variability in the original set. The principal component directions are presented in Section 6.3.1 as directions in feature space along which the original data are *highly variable*. These directions also define lines and subspaces that are *as close as possible* to the data cloud. To perform principal components regression, we simply use principal components as predictors in a regression model in place of the original larger set of variables.

Principal components analysis (PCA) refers to the process by which principal components are computed, and the subsequent use of these components in understanding the data. PCA is an unsupervised approach, since it involves only a set of features $X_1, X_2, \dots, X_p$, and no associated response Y . Apart from producing derived variables for use in supervised learning problems, PCA also serves as a tool for data visualization (visualization of

principal
components
analysis

the observations or visualization of the variables). It can also be used as a tool for data imputation — that is, for filling in missing values in a data matrix.

We now discuss PCA in greater detail, focusing on the use of PCA as a tool for unsupervised data exploration, in keeping with the topic of this chapter.

12.2.1 What Are Principal Components?

Suppose that we wish to visualize n observations with measurements on a set of p features, $X_1, X_2, \dots, X_p$, as part of an exploratory data analysis. We could do this by examining two-dimensional scatterplots of the data, each of which contains the n observations' measurements on two of the features. However, there are $\binom{p}{2} = p(p-1)/2$ such scatterplots; for example, with $p = 10$ there are 45 plots! If p is large, then it will certainly not be possible to look at all of them; moreover, most likely none of them will be informative since they each contain just a small fraction of the total information present in the data set. Clearly, a better method is required to visualize the n observations when p is large. In particular, we would like to find a low-dimensional representation of the data that captures as much of the information as possible. For instance, if we can obtain a two-dimensional representation of the data that captures most of the information, then we can plot the observations in this low-dimensional space.

PCA provides a tool to do just this. It finds a low-dimensional representation of a data set that contains as much as possible of the variation. The idea is that each of the n observations lives in p -dimensional space, but not all of these dimensions are equally interesting. PCA seeks a small number of dimensions that are as interesting as possible, where the concept of *interesting* is measured by the amount that the observations vary along each dimension. Each of the dimensions found by PCA is a linear combination of the p features. We now explain the manner in which these dimensions, or *principal components*, are found.

The *first principal component* of a set of features $X_1, X_2, \dots, X_p$ is the normalized linear combination of the features

$$Z_1 = \phi_{11}X_1 + \phi_{21}X_2 + \cdots + \phi_{p1}X_p \quad (12.1)$$

that has the largest variance. By *normalized*, we mean that $\sum_{j=1}^p \phi_{j1}^2 = 1$. We refer to the elements $\phi_{11}, \dots, \phi_{p1}$ as the *loadings* of the first principal component; together, the loadings make up the principal component loading vector, $\phi_1 = (\phi_{11} \ \phi_{21} \ \dots \ \phi_{p1})^T$. We constrain the loadings so that their sum of squares is equal to one, since otherwise setting these elements to be arbitrarily large in absolute value could result in an arbitrarily large variance. loading

Given an $n \times p$ data set $\mathbf{X}$, how do we compute the first principal component? Since we are only interested in variance, we assume that each of the variables in $\mathbf{X}$ has been centered to have mean zero (that is, the column means of $\mathbf{X}$ are zero). We then look for the linear combination of the sample feature values of the form

$$z_{i1} = \phi_{11}x_{i1} + \phi_{21}x_{i2} + \cdots + \phi_{p1}x_{ip} \quad (12.2)$$

that has largest sample variance, subject to the constraint that $\sum_{j=1}^p \phi_{j1}^2 = 1$. In other words, the first principal component loading vector solves the optimization problem

$$\underset{\phi_{11}, \dots, \phi_{p1}}{\text{maximize}} \left\{ \frac{1}{n} \sum_{i=1}^n \left(\sum_{j=1}^p \phi_{j1} x_{ij} \right)^2 \right\} \text{ subject to } \sum_{j=1}^p \phi_{j1}^2 = 1. \quad (12.3)$$

From (12.2) we can write the objective in (12.3) as $\frac{1}{n} \sum_{i=1}^n z_{i1}^2$. Since $\frac{1}{n} \sum_{i=1}^n x_{ij} = 0$, the average of the $z_{11}, \dots, z_{n1}$ will be zero as well. Hence the objective that we are maximizing in (12.3) is just the sample variance of the n values of z_{i1} . We refer to $z_{11}, \dots, z_{n1}$ as the *scores* of the first principal component. Problem (12.3) can be solved via an *eigen decomposition*, a standard technique in linear algebra, but the details are outside of the scope of this book.¹

score
eigen decomposition

There is a nice geometric interpretation of the first principal component. The loading vector ϕ_1 with elements $\phi_{11}, \phi_{21}, \dots, \phi_{p1}$ defines a direction in feature space along which the data vary the most. If we project the n data points $x_1, \dots, x_n$ onto this direction, the projected values are the principal component scores $z_{11}, \dots, z_{n1}$ themselves. For instance, Figure 6.14 on page 254 displays the first principal component loading vector (green solid line) on an advertising data set. In these data, there are only two features, and so the observations as well as the first principal component loading vector can be easily displayed. As can be seen from (6.19), in that data set $\phi_{11} = 0.839$ and $\phi_{21} = 0.544$.

After the first principal component Z_1 of the features has been determined, we can find the second principal component Z_2 . The second principal component is the linear combination of $X_1, \dots, X_p$ that has maximal variance out of all linear combinations that are *uncorrelated* with Z_1 . The second principal component scores $z_{12}, z_{22}, \dots, z_{n2}$ take the form

$$z_{i2} = \phi_{12}x_{i1} + \phi_{22}x_{i2} + \dots + \phi_{p2}x_{ip}, \quad (12.4)$$

where ϕ_2 is the second principal component loading vector, with elements $\phi_{12}, \phi_{22}, \dots, \phi_{p2}$. It turns out that constraining Z_2 to be uncorrelated with Z_1 is equivalent to constraining the direction ϕ_2 to be orthogonal (perpendicular) to the direction ϕ_1 . In the example in Figure 6.14, the observations lie in two-dimensional space (since $p = 2$), and so once we have found ϕ_1 , there is only one possibility for ϕ_2 , which is shown as a blue dashed line. (From Section 6.3.1, we know that $\phi_{12} = 0.544$ and $\phi_{22} = -0.839$.) But in a larger data set with $p > 2$ variables, there are multiple distinct principal components, and they are defined in a similar manner. To find ϕ_2 , we solve a problem similar to (12.3) with ϕ_2 replacing ϕ_1 , and with the additional constraint that ϕ_2 is orthogonal to ϕ_1 .²

¹As an alternative to the eigen decomposition, a related technique called the singular value decomposition can be used. This will be explored in the lab at the end of this chapter.

²On a technical note, the principal component directions $\phi_1, \phi_2, \phi_3, \dots$ are given by the ordered sequence of eigenvectors of the matrix $\mathbf{X}^T \mathbf{X}$, and the variances of the components are the eigenvalues. There are at most $\min(n-1, p)$ principal components.

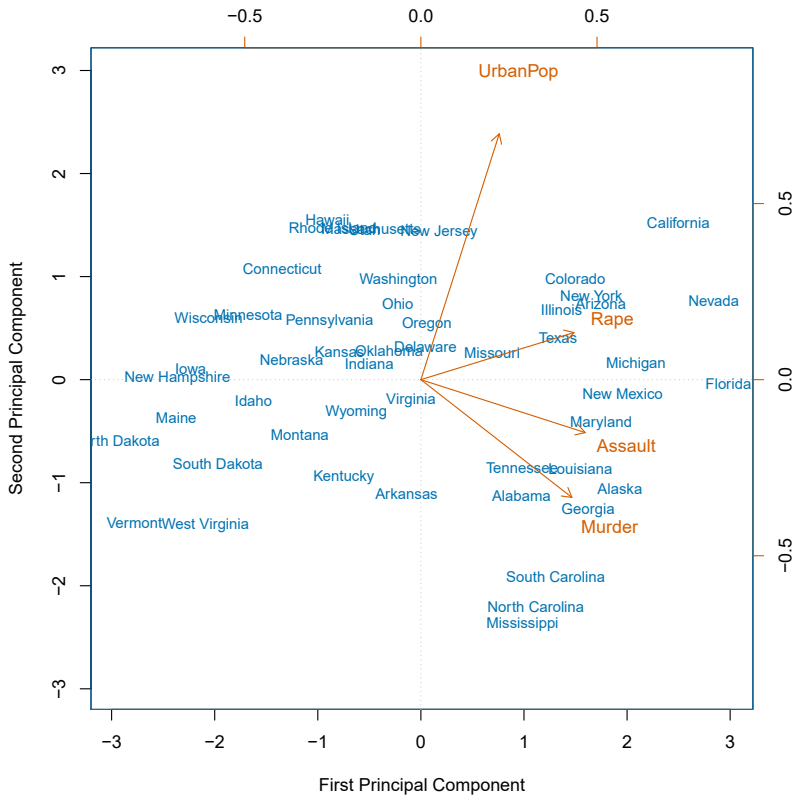


FIGURE 12.1. The first two principal components for the **USArrests** data. The blue state names represent the scores for the first two principal components. The orange arrows indicate the first two principal component loading vectors (with axes on the top and right). For example, the loading for **Rape** on the first component is 0.54, and its loading on the second principal component 0.17 (the word **Rape** is centered at the point (0.54, 0.17)). This figure is known as a biplot, because it displays both the principal component scores and the principal component loadings.

Once we have computed the principal components, we can plot them against each other in order to produce low-dimensional views of the data. For instance, we can plot the score vector Z_1 against Z_2 , Z_1 against Z_3 , Z_2 against Z_3 , and so forth. Geometrically, this amounts to projecting the original data down onto the subspace spanned by ϕ_1 , ϕ_2 , and ϕ_3 , and plotting the projected points.

We illustrate the use of PCA on the **USArrests** data set. For each of the 50 states in the United States, the data set contains the number of arrests per 100,000 residents for each of three crimes: **Assault**, **Murder**, and **Rape**. We also record **UrbanPop** (the percent of the population in each state living in urban areas). The principal component score vectors have length $n = 50$, and the principal component loading vectors have length $p = 4$. PCA was performed after standardizing each variable to have mean zero and standard

	PC1	PC2
Murder	0.5358995	−0.4181809
Assault	0.5831836	−0.1879856
UrbanPop	0.2781909	0.8728062
Rape	0.5434321	0.1673186

TABLE 12.1. The principal component loading vectors, ϕ_1 and ϕ_2 , for the **USArrests** data. These are also displayed in Figure 12.1.

deviation one. Figure 12.1 plots the first two principal components of these data. The figure represents both the principal component scores and the loading vectors in a single *biplot* display. The loadings are also given in Table 12.2.1. biplot

In Figure 12.1, we see that the first loading vector places approximately equal weight on **Assault**, **Murder**, and **Rape**, but with much less weight on **UrbanPop**. Hence this component roughly corresponds to a measure of overall rates of serious crimes. The second loading vector places most of its weight on **UrbanPop** and much less weight on the other three features. Hence, this component roughly corresponds to the level of urbanization of the state. Overall, we see that the crime-related variables (**Murder**, **Assault**, and **Rape**) are located close to each other, and that the **UrbanPop** variable is far from the other three. This indicates that the crime-related variables are correlated with each other—states with high murder rates tend to have high assault and rape rates—and that the **UrbanPop** variable is less correlated with the other three.

We can examine differences between the states via the two principal component score vectors shown in Figure 12.1. Our discussion of the loading vectors suggests that states with large positive scores on the first component, such as California, Nevada and Florida, have high crime rates, while states like North Dakota, with negative scores on the first component, have low crime rates. California also has a high score on the second component, indicating a high level of urbanization, while the opposite is true for states like Mississippi. States close to zero on both components, such as Indiana, have approximately average levels of both crime and urbanization.

12.2.2 Another Interpretation of Principal Components

The first two principal component loading vectors in a simulated three-dimensional data set are shown in the left-hand panel of Figure 12.2; these two loading vectors span a plane along which the observations have the highest variance.

In the previous section, we describe the principal component loading vectors as the directions in feature space along which the data vary the most, and the principal component scores as projections along these directions. However, an alternative interpretation of principal components can also be

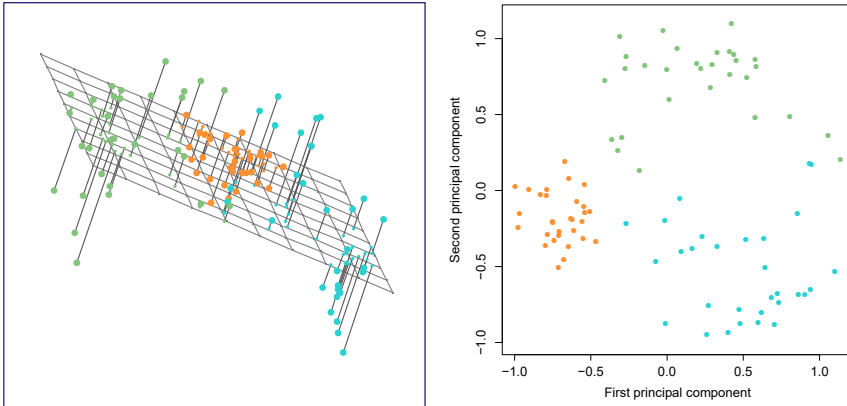


FIGURE 12.2. Ninety observations simulated in three dimensions. The observations are displayed in color for ease of visualization. Left: the first two principal component directions span the plane that best fits the data. The plane is positioned to minimize the sum of squared distances to each point. Right: the first two principal component score vectors give the coordinates of the projection of the 90 observations onto the plane.

useful: principal components provide low-dimensional linear surfaces that are *closest* to the observations. We expand upon that interpretation here.³

The first principal component loading vector has a very special property: it is the line in p -dimensional space that is *closest* to the n observations (using average squared Euclidean distance as a measure of closeness). This interpretation can be seen in the left-hand panel of Figure 6.15; the dashed lines indicate the distance between each observation and the line defined by the first principal component loading vector. The appeal of this interpretation is clear: we seek a single dimension of the data that lies as close as possible to all of the data points, since such a line will likely provide a good summary of the data.

The notion of principal components as the dimensions that are closest to the n observations extends beyond just the first principal component. For instance, the first two principal components of a data set span the plane that is closest to the n observations, in terms of average squared Euclidean distance. An example is shown in the left-hand panel of Figure 12.2. The first three principal components of a data set span the three-dimensional hyperplane that is closest to the n observations, and so forth.

Using this interpretation, together the first M principal component score vectors and the first M principal component loading vectors provide the best M -dimensional approximation (in terms of Euclidean distance) to

³In this section, we continue to assume that each column of the data matrix $\mathbf{X}$ has been centered to have mean zero—that is, the column mean has been subtracted from each column.

the i th observation x_{ij} . This representation can be written as

$$x_{ij} \approx \sum_{m=1}^M z_{im} \phi_{jm}. \quad (12.5)$$

We can state this more formally by writing down an optimization problem. Suppose the data matrix $\mathbf{X}$ is column-centered. Out of all approximations of the form $x_{ij} \approx \sum_{m=1}^M a_{im} b_{jm}$, we could ask for the one with the smallest residual sum of squares:

$$\underset{\mathbf{A} \in \mathbb{R}^{n \times M}, \mathbf{B} \in \mathbb{R}^{p \times M}}{\text{minimize}} \left\{ \sum_{j=1}^p \sum_{i=1}^n \left(x_{ij} - \sum_{m=1}^M a_{im} b_{jm} \right)^2 \right\}. \quad (12.6)$$

Here, $\mathbf{A}$ is an $n \times M$ matrix whose (i, m) element is a_{im} , and $\mathbf{B}$ is a $p \times M$ element whose (j, m) element is b_{jm} .

It can be shown that for any value of M , the columns of the matrices $\hat{\mathbf{A}}$ and $\hat{\mathbf{B}}$ that solve (12.6) are in fact the first M principal components score and loading vectors. In other words, if $\hat{\mathbf{A}}$ and $\hat{\mathbf{B}}$ solve (12.6), then $\hat{a}_{im} = z_{im}$ and $\hat{b}_{jm} = \phi_{jm}$.⁴ This means that the smallest possible value of the objective in (12.6) is

$$\sum_{j=1}^p \sum_{i=1}^n \left(x_{ij} - \sum_{m=1}^M z_{im} \phi_{jm} \right)^2. \quad (12.7)$$

In summary, together the M principal component score vectors and M principal component loading vectors can give a good approximation to the data when M is sufficiently large. When $M = \min(n - 1, p)$, then the representation is exact: $x_{ij} = \sum_{m=1}^M z_{im} \phi_{jm}$.

12.2.3 The Proportion of Variance Explained

In Figure 12.2, we performed PCA on a three-dimensional data set (left-hand panel) and projected the data onto the first two principal component loading vectors in order to obtain a two-dimensional view of the data (i.e. the principal component score vectors; right-hand panel). We see that this two-dimensional representation of the three-dimensional data does successfully capture the major pattern in the data: the orange, green, and cyan observations that are near each other in three-dimensional space remain nearby in the two-dimensional representation. Similarly, we have seen on the **USArrests** data set that we can summarize the 50 observations and 4 variables using just the first two principal component score vectors and the first two principal component loading vectors.

We can now ask a natural question: how much of the information in a given data set is lost by projecting the observations onto the first few principal components? That is, how much of the variance in the data is *not* contained in the first few principal components? More generally, we are interested in knowing the *proportion of variance explained* (PVE) by each

proportion
of variance
explained

⁴Technically, the solution to (12.6) is not unique. Thus, it is more precise to state that any solution to (12.6) can be easily transformed to yield the principal components.

principal component. The *total variance* present in a data set (assuming that the variables have been centered to have mean zero) is defined as

$$\sum_{j=1}^p \text{Var}(X_j) = \sum_{j=1}^p \frac{1}{n} \sum_{i=1}^n x_{ij}^2, \quad (12.8)$$

and the variance explained by the m th principal component is

$$\frac{1}{n} \sum_{i=1}^n z_{im}^2 = \frac{1}{n} \sum_{i=1}^n \left(\sum_{j=1}^p \phi_{jm} x_{ij} \right)^2. \quad (12.9)$$

Therefore, the PVE of the m th principal component is given by

$$\frac{\sum_{i=1}^n z_{im}^2}{\sum_{j=1}^p \sum_{i=1}^n x_{ij}^2} = \frac{\sum_{i=1}^n \left(\sum_{j=1}^p \phi_{jm} x_{ij} \right)^2}{\sum_{j=1}^p \sum_{i=1}^n x_{ij}^2}. \quad (12.10)$$

The PVE of each principal component is a positive quantity. In order to compute the cumulative PVE of the first M principal components, we can simply sum (12.10) over each of the first M PVEs. In total, there are $\min(n-1, p)$ principal components, and their PVEs sum to one.

In Section 12.2.2, we showed that the first M principal component loading and score vectors can be interpreted as the best M -dimensional approximation to the data, in terms of residual sum of squares. It turns out that the variance of the data can be decomposed into the variance of the first M principal components plus the mean squared error of this M -dimensional approximation, as follows:

$$\underbrace{\sum_{j=1}^p \frac{1}{n} \sum_{i=1}^n x_{ij}^2}_{\text{Var. of data}} = \underbrace{\sum_{m=1}^M \frac{1}{n} \sum_{i=1}^n z_{im}^2}_{\text{Var. of first } M \text{ PCs}} + \underbrace{\frac{1}{n} \sum_{j=1}^p \sum_{i=1}^n \left(x_{ij} - \sum_{m=1}^M z_{im} \phi_{jm} \right)^2}_{\text{MSE of } M\text{-dimensional approximation}} \quad (12.11)$$

The three terms in this decomposition are discussed in (12.8), (12.9), and (12.7), respectively. Since the first term is fixed, we see that by maximizing the variance of the first M principal components, we minimize the mean squared error of the M -dimensional approximation, and vice versa. This explains why principal components can be equivalently viewed as minimizing the approximation error (as in Section 12.2.2) or maximizing the variance (as in Section 12.2.1).

Moreover, we can use (12.11) to see that the PVE defined in (12.10) equals

$$1 - \frac{\sum_{j=1}^p \sum_{i=1}^n \left(x_{ij} - \sum_{m=1}^M z_{im} \phi_{jm} \right)^2}{\sum_{j=1}^p \sum_{i=1}^n x_{ij}^2} = 1 - \frac{\text{RSS}}{\text{TSS}},$$

where TSS represents the total sum of squared elements of $\mathbf{X}$, and RSS represents the residual sum of squares of the M -dimensional approximation given by the principal components. Recalling the definition of R^2 from (3.17), this means that we can interpret the PVE as the R^2 of the approximation for $\mathbf{X}$ given by the first M principal components.

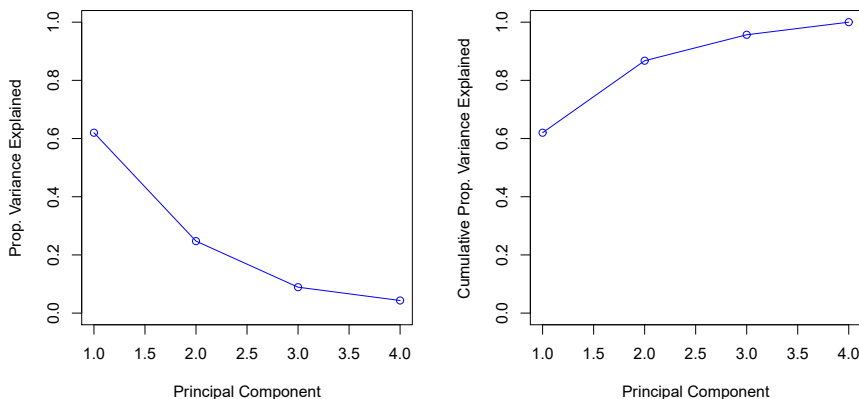


FIGURE 12.3. Left: a *scree plot* depicting the proportion of variance explained by each of the four principal components in the `USArrests` data. Right: the cumulative proportion of variance explained by the four principal components in the `USArrests` data.

In the `USArrests` data, the first principal component explains 62.0% of the variance in the data, and the next principal component explains 24.7% of the variance. Together, the first two principal components explain almost 87% of the variance in the data, and the last two principal components explain only 13% of the variance. This means that Figure 12.1 provides a pretty accurate summary of the data using just two dimensions. The PVE of each principal component, as well as the cumulative PVE, is shown in Figure 12.3. The left-hand panel is known as a *scree plot*, and will be discussed later in this chapter.

scree plot

12.2.4 More on PCA

Scaling the Variables

We have already mentioned that before PCA is performed, the variables should be centered to have mean zero. Furthermore, *the results obtained when we perform PCA will also depend on whether the variables have been individually scaled* (each multiplied by a different constant). This is in contrast to some other supervised and unsupervised learning techniques, such as linear regression, in which scaling the variables has no effect. (In linear regression, multiplying a variable by a factor of c will simply lead to multiplication of the corresponding coefficient estimate by a factor of $1/c$, and thus will have no substantive effect on the model obtained.)

For instance, Figure 12.1 was obtained after scaling each of the variables to have standard deviation one. This is reproduced in the left-hand plot in Figure 12.4. Why does it matter that we scaled the variables? In these data, the variables are measured in different units; `Murder`, `Rape`, and `Assault` are reported as the number of occurrences per 100,000 people, and `UrbanPop` is the percentage of the state's population that lives in an urban area. These four variables have variances of 18.97, 87.73, 6945.16, and 209.5, respectively. Consequently, if we perform PCA on the unscaled variables, then

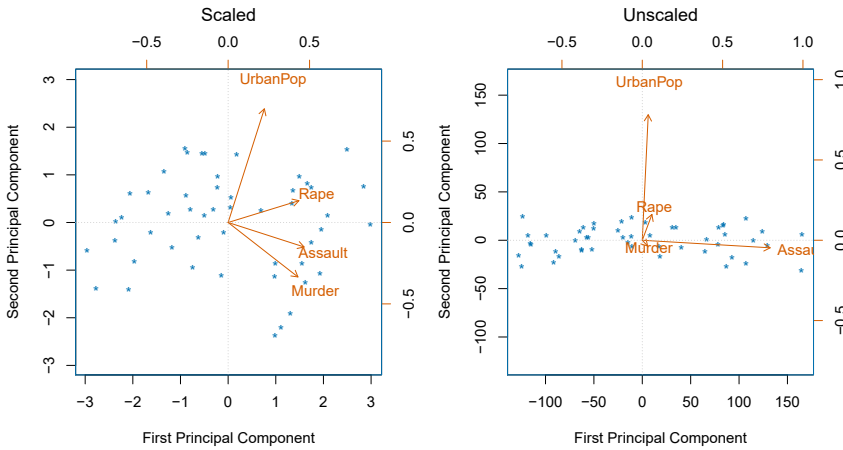


FIGURE 12.4. Two principal component biplots for the `USArrests` data. Left: the same as Figure 12.1, with the variables scaled to have unit standard deviations. Right: principal components using unscaled data. `Assault` has by far the largest loading on the first principal component because it has the highest variance among the four variables. In general, scaling the variables to have standard deviation one is recommended.

the first principal component loading vector will have a very large loading for `Assault`, since that variable has by far the highest variance. The right-hand plot in Figure 12.4 displays the first two principal components for the `USArrests` data set, without scaling the variables to have standard deviation one. As predicted, the first principal component loading vector places almost all of its weight on `Assault`, while the second principal component loading vector places almost all of its weight on `UrbanPop`. Comparing this to the left-hand plot, we see that scaling does indeed have a substantial effect on the results obtained.

However, this result is simply a consequence of the scales on which the variables were measured. For instance, if `Assault` were measured in units of the number of occurrences per 100 people (rather than number of occurrences per 100,000 people), then this would amount to dividing all of the elements of that variable by 1,000. Then the variance of the variable would be tiny, and so the first principal component loading vector would have a very small value for that variable. Because it is undesirable for the principal components obtained to depend on an arbitrary choice of scaling, we typically scale each variable to have standard deviation one before we perform PCA.

In certain settings, however, the variables may be measured in the same units. In this case, we might not wish to scale the variables to have standard deviation one before performing PCA. For instance, suppose that the variables in a given data set correspond to expression levels for p genes. Then since expression is measured in the same “units” for each gene, we might choose not to scale the genes to each have standard deviation one.

Uniqueness of the Principal Components

While in theory the principal components need not be unique, in almost all practical settings they are (up to sign flips). This means that two different software packages will yield the same principal component loading vectors, although the signs of those loading vectors may differ. The signs may differ because each principal component loading vector specifies a direction in p -dimensional space: flipping the sign has no effect as the direction does not change. (Consider Figure 6.14—the principal component loading vector is a line that extends in either direction, and flipping its sign would have no effect.) Similarly, the score vectors are unique up to a sign flip, since the variance of Z is the same as the variance of $-Z$. It is worth noting that when we use (12.5) to approximate x_{ij} we multiply z_{im} by ϕ_{jm} . Hence, if the sign is flipped on both the loading and score vectors, the final product of the two quantities is unchanged.

Deciding How Many Principal Components to Use

In general, an $n \times p$ data matrix $\mathbf{X}$ has $\min(n-1, p)$ distinct principal components. However, we usually are not interested in all of them; rather, we would like to use just the first few principal components in order to visualize or interpret the data. In fact, we would like to use the smallest number of principal components required to get a *good* understanding of the data. How many principal components are needed? Unfortunately, there is no single (or simple!) answer to this question.

We typically decide on the number of principal components required to visualize the data by examining a *scree plot*, such as the one shown in the left-hand panel of Figure 12.3. We choose the smallest number of principal components that are required in order to explain a sizable amount of the variation in the data. This is done by eyeballing the scree plot, and looking for a point at which the proportion of variance explained by each subsequent principal component drops off. This drop is often referred to as an *elbow* in the scree plot. For instance, by inspection of Figure 12.3, one might conclude that a fair amount of variance is explained by the first two principal components, and that there is an elbow after the second component. After all, the third principal component explains less than ten percent of the variance in the data, and the fourth principal component explains less than half that and so is essentially worthless.

However, this type of visual analysis is inherently *ad hoc*. Unfortunately, there is no well-accepted objective way to decide how many principal components are *enough*. In fact, the question of how many principal components are enough is inherently ill-defined, and will depend on the specific area of application and the specific data set. In practice, we tend to look at the first few principal components in order to find interesting patterns in the data. If no interesting patterns are found in the first few principal components, then further principal components are unlikely to be of interest. Conversely, if the first few principal components are interesting, then we typically continue to look at subsequent principal components until no further interesting patterns are found. This is admittedly a subjective ap-